PETRA: CLOUD-BASED AI-POWERED PETITION MANAGEMENT AND MONITORING SYSTEM

CLOUD COMPUTING

Submi&ed by

GOKULASARATHY P S (230701095)
HIMESHWAR N (230701116)
JAYASUDHAN V (230701131)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

MAY 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**PETRA: CLOUD-BASED AI-POWERED PETITION MANAGEMENT AND MONITORING SYSTEM**" is the bonafide work of **GOKULASARATHY P S (230701095), HIMESHWAR N (230701116), JAYASUDHAN V (230701131)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science and Engineering
Rajalakshmi Engineering College

SIGNATURE
Dr. S. Ponmani
Supervisor
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. S. PONMANI M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

GOKULASARATHY P S
HIMESHWAR N
JAYASUDHAN V

ABSTRACT

Existing petition and complaint management systems are mostly manual, inefficient, and lack centralized digital processing. Users face difficulties in submitting petitions with supporting documents, tracking request status, and receiving timely updates. Administrators also struggle to manage large numbers of requests due to the absence of automation, intelligent classification, and priority-based handling mechanisms. Critical issues are often delayed because traditional systems lack analytics, real-time monitoring, secure authentication, and scalable infrastructure.

PETRA – Petition Management System is a cloud-based AI-powered platform developed to modernize petition handling through automation, centralized management, and intelligent processing. The platform enables users to securely submit petitions with PDF and audio support, track request progress in real-time, and receive automated notifications regarding approvals and updates. Administrators can efficiently review, classify, prioritize, and process petitions through a centralized dashboard integrated with analytics and monitoring tools.

The proposed solution uses Microsoft Azure cloud services with a Platform-as-a-Service (PaaS) architecture to ensure scalability, reliability, and high availability. The frontend is developed using React, while the backend uses Flask APIs hosted on Azure App Service. Azure Functions provide serverless background processing, AI-based classification, scheduled reminders, and automated notifications. Azure SQL Database manages structured petition data, and Azure Blob Storage securely stores uploaded files such as PDFs and audio recordings. Azure AD B2C ensures secure user authentication and identity management.

The platform also integrates Natural Language Processing (NLP) models such as Scikit-learn, NLTK, and spaCy for intelligent petition classification and priority scoring. These AI models analyze petition content, identify urgent requests, and automate the prioritization process to improve response time and administrative efficiency. Power BI Embedded dashboards provide real-time analytics, monitoring insights, and data-driven reporting capabilities for better decision-making and operational transparency.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Existing petition and complaint management systems are manual, inefficient, and lack a centralized platform for submitting requests with supporting documents such as PDFs and audio files. Users experience difficulties in tracking petition status, while administrators struggle to manage large numbers of requests due to lack of automation and intelligent prioritization.

Traditional systems also fail to provide:

- Real-time notifications and updates.
- Secure authentication and access control.
- AI-based classification and priority handling.
- Efficient background processing.
- Analytics and monitoring dashboards.
- Centralized storage and management.

Critical petitions such as healthcare or emergency requests are often delayed because existing systems cannot prioritize issues effectively. These limitations result in operational inefficiencies, reduced transparency, poor decision-making, and low user satisfaction.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of PETRA is to design and implement a cloud-based AI-powered petition management platform capable of automating petition submission, classification, prioritization, monitoring, and approval workflows.

The objectives include:

- Provide a centralized digital platform for petition handling.
- Enable secure authentication using Azure AD B2C.
- Support PDF and audio file uploads.
- Implement AI-based classification and priority scoring.
- Provide real-time tracking and automated notifications.
- Improve transparency and operational efficiency.
- Enable data-driven insights through analytics dashboards.
- Ensure scalability and reliability using Microsoft Azure cloud services.

1.3 SCOPE AND BOUNDARIES

The project scope includes:

- User registration and authentication.
- Petition submission with document uploads.
- Petition classification and priority handling.
- Real-time petition tracking.
- Notifications through email and alerts.
- Background job processing.
- Analytics and reporting dashboards.
- Cloud deployment and monitoring.

The project boundaries include:

- The system is limited to petition and complaint management.
- Financial transaction modules are not included.
- Offline petition processing is outside the project scope.

1.4 STAKEHOLDERS AND END USERS

Stakeholders

- Project Development Team
- Institution Administrators
- Government or Organizational Authorities
- Cloud Infrastructure Team

End Users

- Students and citizens submitting petitions.
- Administrators processing requests.
- Department authorities reviewing petitions.
- Monitoring teams analyzing reports and insights

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

React	For building frontend user interface of marketplace
Flutter	Cross-platform Interface Support
T**ailwind**	For important user interface components

Table 2: Backend Technologies

Python Flask	REST API Development
REST APIs	Business Logic and Workflow
NLP Models (Scikit-learn / NLTK / spaCy)	AI Classification

Table 3: Cloud and Infrastructure

Microsoft Azure	The primary cloud provider for hosting and managing services.
Azure App Service	Hosting Frontend and Backend
Azure Functions	Serverless Processing
Azure SQL Database	Structured Data Storage
Azure Blob Storage	PDF and Audio Storage
Terraform	For IaaC provision
Docker	For containerizing applications.
GitHub Actions	CI/CD Automation
Power BI Embedded	Analytics and Reporting
Azure AD B2C	Authentication and Identity Management
Application Insights	Monitoring and Logging

Table 4: AI and Machine Learning

Scikit-learn	Used for petition classification and priority prediction.
NLTK	Used for natural language processing and text preprocessing.

Table 5: DevOps and CI/CD

GitHub Actions	For automating workflows, builds, and deployments.
Docker Hub	As a registry for storing and distributing container images.
Azure CLI	For command-line management of Azure resources.

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including requirements, solution overview, cloud strategy, infrastructure specifications, and service mappings.

Chapter 3 delves into the DevOps implementation, covering CI/CD pipelines, Terraform automation, Docker containerization, orchestration principles, and AI service integration. **Chapter 4** focuses on operational excellence and security, discussing DevSecOps practices, monitoring, access control, and deployment strategies.

Chapter 5 presents the results of the implementation, including summaries, challenges overcome, performance metrics, cost analysis, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

The document concludes with references and appendices containing configuration details and system screenshots.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The PETRA system is designed with multiple cloud-based components to ensure scalability, reliability, and efficient petition management. The user interface is developed for petition submission and tracking and is hosted using Azure App Service with auto-scaling based on CPU usage and incoming requests. The API and backend services handle business logic, authentication, and petition workflows through Azure App Service with horizontal scaling support. AI-based petition classification and prioritization are implemented using Azure Functions with event-driven scaling to efficiently process requests. Azure SQL Database is used to store user details, petition records, and status information with vertical scaling support for improved performance. Uploaded files such as PDFs and audio recordings are securely stored in Azure Blob Storage with automatic storage scaling. Secure authentication and user identity management are handled through Azure AD B2C with managed scaling provided by Azure. Notification services including Email and SMS alerts are processed using Azure Functions with event-based scaling. Background tasks such as reminders, queue handling, and scheduled jobs are managed using Queue Storage integrated with Azure Functions. Analytics dashboards and reporting functionalities are implemented using Power BI Embedded, which supports scaling through capacity tiers. Monitoring, logging, and system health tracking are performed using Application Insights with auto-scaling support integrated with app services.

Non-Functional Requirements

The PETRA system is designed to achieve high availability with a target uptime of more than 99.9% using the Azure App Service SLA, ensuring reliable access to users at all times. The platform is optimized for high performance with response times below 2 seconds through Azure App Service integrated with Content Delivery Network (CDN) support, providing a fast and smooth user experience. Scalability is achieved by supporting more than 1000 concurrent users using auto-scaling cloud services that dynamically handle increasing workloads efficiently. Security is maintained using Azure AD B2C, HTTPS communication, and encrypted data transfer to protect user information and ensure secure authentication. Data durability is ensured with 99.999% reliability through Azure SQL Database backups and Azure Blob Storage redundancy, preventing data loss and supporting disaster recovery. Reliability is further improved using fault-tolerant Azure Functions that

enable continuous system operation and efficient background processing. Maintainability is simplified through Azure DevOps tools and monitoring services, allowing easy updates, debugging, and deployment management. The system also focuses on usability by providing a user-friendly interface developed using React, improving accessibility and user satisfaction. Notification latency is designed to be near real-time through event-driven Azure Functions, ensuring timely alerts and updates to users and administrators. In addition, analytics accuracy is enhanced using Power BI Embedded dashboards, supporting data-driven decision-making and efficient monitoring of petition management activities.

2.2 PROPOSED SOLUTION OVERVIEWS

PETRA is designed as a cloud-based AI-powered petition management platform that provides centralized petition submission, intelligent processing, and real-time tracking facilities. The system enables users to securely submit petitions with supporting documents while administrators can efficiently manage, classify, and prioritize requests through a centralized dashboard. The frontend of the application is developed using React and Flutter to provide an interactive and user-friendly interface for petition submission and monitoring. Backend services and business logic are implemented using Flask APIs, which handle authentication, petition workflows, notifications, and request management.\n\nAzure Functions are integrated to perform AI-based petition classification, background processing, and automated notification services. Petition data, user information, and status details are securely stored in Azure SQL Database, while uploaded PDF and audio files are maintained in Azure Blob Storage for scalable and reliable storage management. Power BI Embedded is used to generate analytics dashboards and reporting insights that help administrators make data-driven decisions. Secure authentication and identity management are implemented using Azure AD B2C to ensure protected access to the platform.\n\nThe system also integrates Natural Language Processing (NLP) models such as Scikit-learn, NLTK, and spaCy for intelligent petition classification and priority scoring. These AI models analyze petition content, categorize requests, and identify urgent issues to ensure that high-priority petitions are handled quickly and efficiently.

2.3 CLOUD DEPLOYMENT STATERGY

The PETRA system is deployed using Microsoft Azure cloud services with a cloud-native and scalable architecture. The deployment strategy primarily follows the Platform-as-a-Service (PaaS) model to reduce infrastructure management complexity while ensuring high availability, reliability, and operational efficiency. The application is deployed in the

Central India Azure region to achieve low latency, better performance, and reliable service availability for users.

The frontend application developed using React and Flutter is hosted on Azure App Service, providing secure and scalable web hosting with automatic scaling based on incoming traffic and CPU utilization. The backend services developed using Flask APIs are also deployed through Azure App Service, enabling efficient request handling, authentication, and petition processing.

Azure Functions are used for serverless execution of AI-based petition classification, automated notifications, scheduled reminders, and background processing tasks. This event-driven architecture ensures efficient resource utilization and cost optimization by allocating compute resources only when required.

Azure SQL Database is used for storing structured data such as user information, petition records, petition status, and logs. Uploaded files including PDFs and audio recordings are securely stored in Azure Blob Storage, which provides scalable and reliable cloud storage with backup and redundancy features.

Secure user authentication and identity management are implemented using Azure AD B2C, enabling role-based access control and protected system access. Power BI Embedded is integrated into the system to provide analytics dashboards and real-time reporting for monitoring petition activities and system performance.

The deployment workflow is automated using GitHub Actions and Docker-based containerization. Continuous Integration and Continuous Deployment (CI/CD) pipelines automate application building, testing, and deployment processes, ensuring faster releases and reduced manual effort. Monitoring and observability are implemented using Azure Application Insights for tracking application performance, logs, errors, and system health in real time.

2.4 INFRASTRUCTURE REQUIREMENTS

The PETRA system requires a scalable and reliable cloud infrastructure to support petition submission, AI-based processing, file storage, analytics, and real-time monitoring. Microsoft Azure cloud services are used to provide high availability, security, and operational efficiency.

Compute Resources:

The frontend and backend applications are hosted using Azure App Service, which provides scalable web hosting with automatic scaling based on CPU utilization and incoming requests. Azure Functions are used for serverless execution of AI-based petition classification, notifications, background jobs, and scheduled processing tasks. The platform supports dynamic scaling to efficiently manage varying workloads and concurrent users.

Storage Requirements:

Azure SQL Database is used to store structured data such as user information, petition records, status updates, logs, and analytics data. The database supports backup, recovery, and high availability features for reliable data management. Azure Blob Storage is used to store uploaded PDF documents, audio files, and other supporting media files securely with scalable storage capacity and redundancy support.

Networking Requirements:

The system uses secure HTTPS communication for all frontend and backend interactions to ensure encrypted data transfer. Azure networking services and firewall rules are configured to provide secure access to APIs, databases, and storage services while protecting the platform from unauthorized access.

Authentication and Security Infrastructure:

Azure AD B2C is integrated to provide secure authentication, identity management, and role-based access control for users and administrators. JWT-based authentication and secure API endpoints ensure protected access to petition management functionalities.

Monitoring and Analytics Infrastructure:

Azure Application Insights is used for monitoring application performance, logs, request tracking, and error analysis in real time. Power BI Embedded is integrated to generate analytics dashboards and reporting insights for administrators and monitoring teams.

DevOps and Deployment Infrastructure:

GitHub Actions and Docker are used for CI/CD automation and containerized deployment. Automated workflows manage application building, testing, deployment, and infrastructure updates, ensuring consistent and reliable cloud deployment processes.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	Azure Service	Purpose	Est. Monthly Cost (₹)
Frontend & Backend	Azure App Service	Host UI and APIs	₹1,500 – ₹4,000
Database	Azure SQL Database	Store petitions and users	₹1,000 – ₹3,000
File Storage	Azure Blob Storage	Store PDFs and audio	₹200 – ₹800
Authentication	Azure AD B2C	User identity management	Free – ₹500
AI Processing	Azure Functions	AI classification	₹200 – ₹1,000
Notifications	Azure Functions + Twilio	Alerts and updates	₹300 – ₹1,500
Background Tasks	Azure Functions	Scheduled processing	₹200 – ₹800
Analytics	Power BI Embedded	Dashboards and reports	₹5,000 – ₹15,000
Monitoring	Application Insights	Logs and monitoring	₹200 – ₹1,000
Backup	Azure Backup	Data recovery	₹300 – ₹1,000

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The project uses a multi-stage CI/CD pipeline built with GitHub Actions to automate the deployment. This setup uses Infrastructure as Code (IaC) and containerization to deploy components across various Azure services.

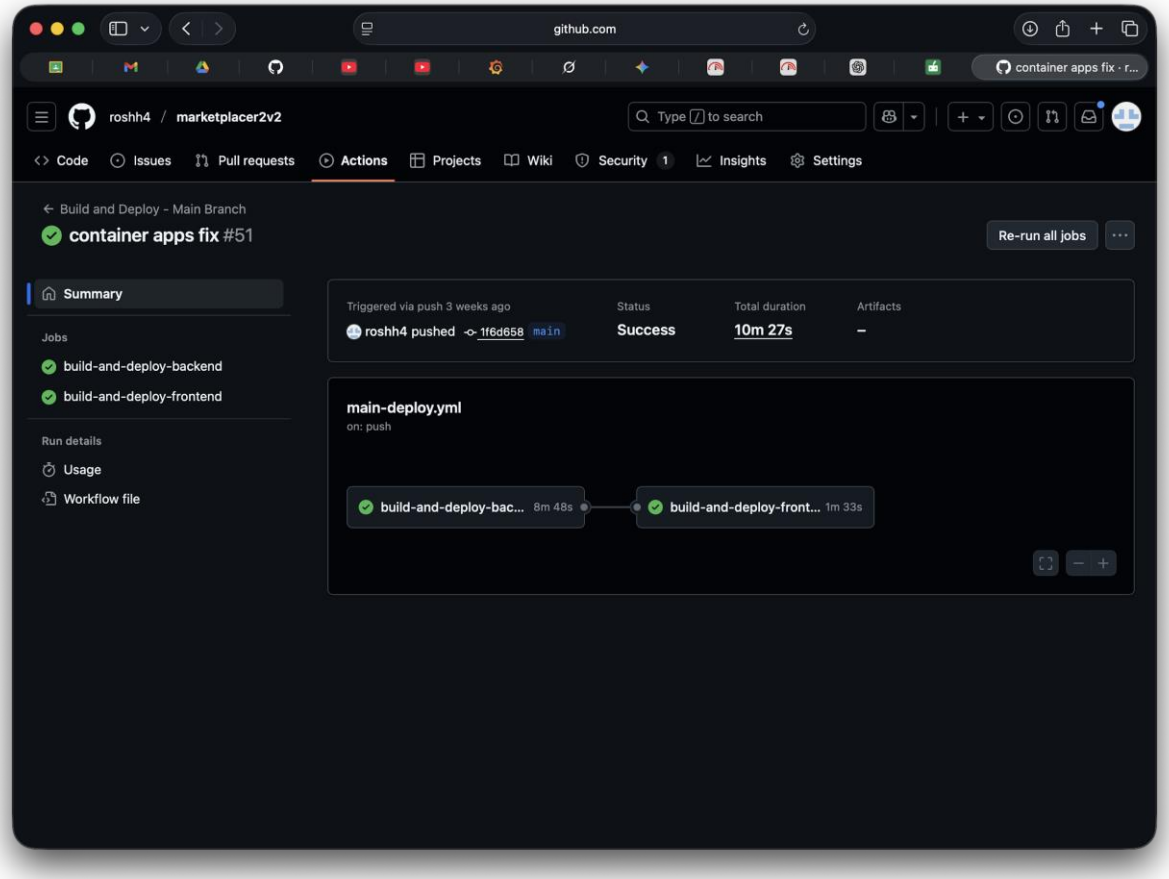


Figure 1. Github Actions Workflow

The pipeline follows 2-stage deployment pattern:

1. **Backend:** Deploys the main application (as a Container App).
2. **Frontend:** Builds and deploys the static web app (React).

The entire process is triggered either by a push to the main branch or through a manual workflow dispatch.

Pipeline Stages in Detail:

Stage 1: Backend Deployment (build-and-deploy-backend)

1. **Versioning and Containers:** A unique version tag is created for the container image using the date and github commit ID (e.g., v20251029-a1b2c3d). The image is then built and pushed to the registry.
2. **Infrastructure as Code (IaC):** Terraform manages all Azure resources. It ensures that the infrastructure is consistent and changes are applied.
3. **Azure Deployment:** Secure access to Azure is handled through a Service Principal using secrets stored in GitHub. The new container image is deployed to Azure Container Apps using a zero-downtime rolling update.
4. **Health Check:** After deployment, the pipeline waits and performs an automated check against the backend's /healthendpoint to confirm the service is running correctly before moving on.

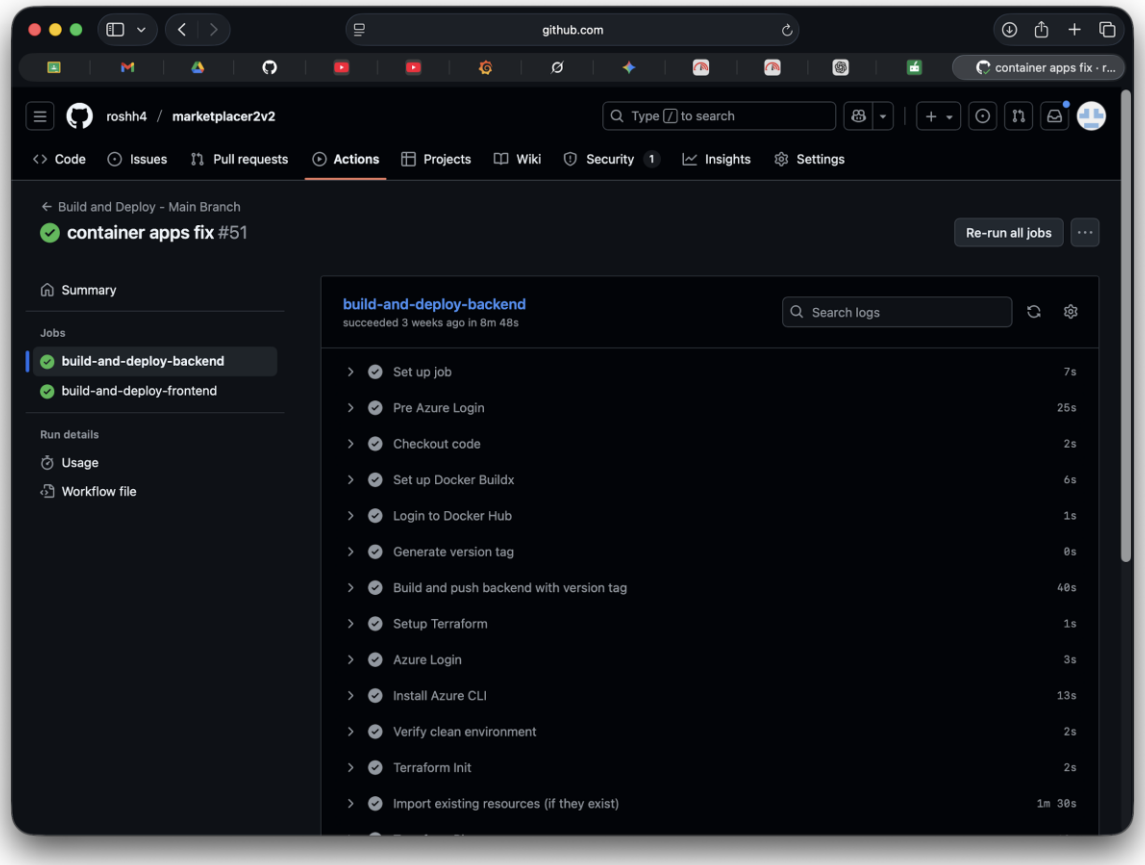


Figure 2. Backend Deployment Job Run

Stage 2: Frontend Deployment (build-and-deploy-frontend)

Dynamic Configuration: The URL of the newly deployed Backend is automatically passed to the frontend build process. This ensures the React application is configured with the correct endpoints for the current deployment.

Hosting: The optimized frontend build is deployed to Azure Static Web Apps (SWA).

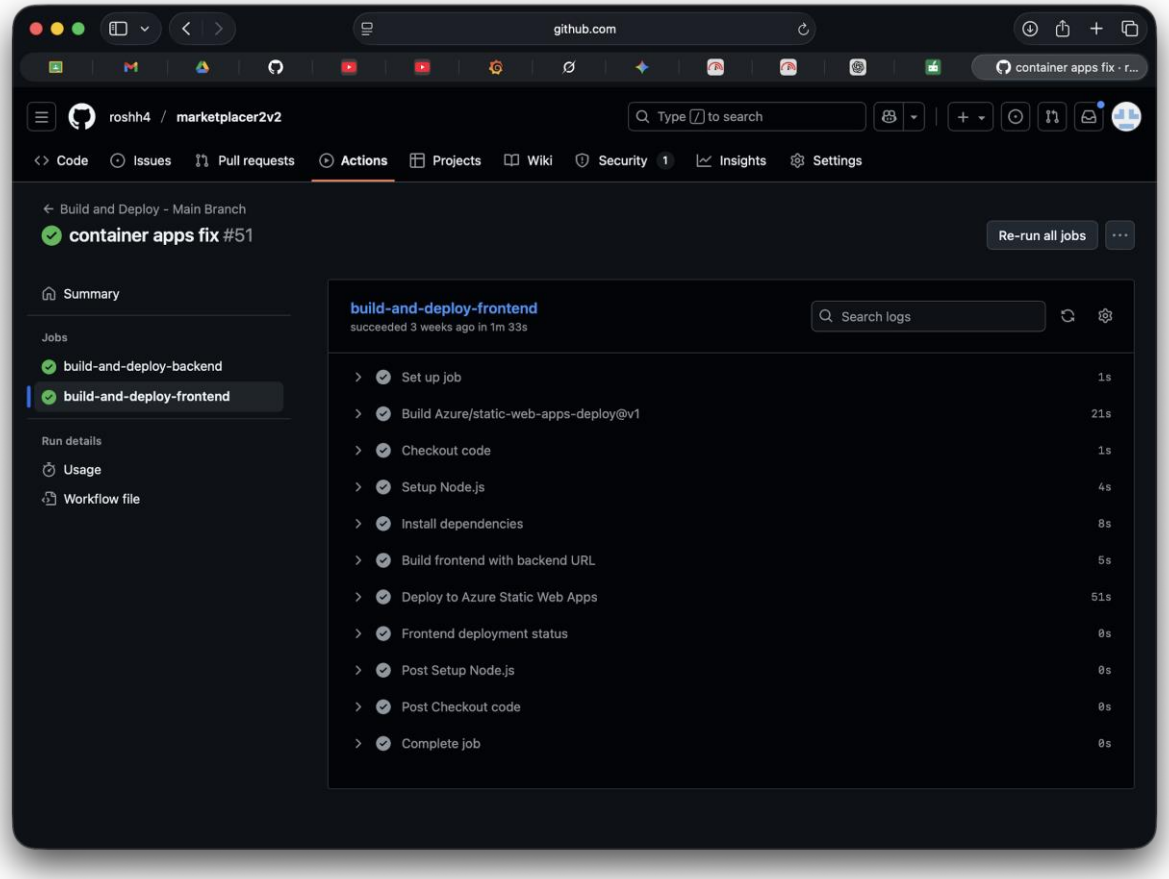


Figure 3. Frontend Deployment Job Run

Required GitHub Secrets

Table 7. List of Github Secrets

1	AZURE_CLIENT_ID
2	AZURE_CLIENT_SECRET
3	AZURE_STATIC_WEB_APPS_API_TOKEN
4	AZURE_STORAGE_ACCOUNT_NAME
5	AZURE_SUBSCRIPTION_ID
6	AZURE_TENANT_ID
7	CONTENT_SAFETY_KEY
8	DB_PASSWORD
9	DOCKERHUB_TOKEN
10	DOCKERHUB_USERNAME
11	GEMINI_API_KEY

3.2 TERRAFORM INFRASTRUCTURE-AS-CODE (IaC)

Terraform is used in the PETRA system to automate the provisioning and management of Azure cloud resources through Infrastructure-as-Code (IaC). Instead of manually creating cloud services through the Azure portal, Terraform allows the entire infrastructure to be defined using configuration files. This approach improves consistency, scalability, automation, and deployment reliability.

The Terraform configuration is mainly organized into three files:

main.tf – Contains definitions for Azure resources such as App Service, Azure Functions, SQL Database, Blob Storage, and monitoring services.

variables.tf – Stores configurable input variables such as resource names, regions, database credentials, and environment settings.

outputs.tf – Defines output values such as application URLs, database endpoints, and storage account details used during deployment.

Using Terraform, the PETRA platform automatically provisions all required Azure services including:

Azure App Service for hosting frontend and backend applications.

Azure Functions for AI classification, notifications, and background processing.

Azure SQL Database for storing user and petition data.

Azure Blob Storage for storing PDF and audio files.

Azure AD B2C integration for secure authentication.

Application Insights for monitoring and logging.

Terraform ensures that infrastructure deployment remains consistent across development, testing, and production environments. Any infrastructure changes can be tracked through version control systems such as GitHub, making deployment management easier and more reliable.

The Terraform workflow in PETRA follows these steps:

Initialize Terraform using terraform init.

Validate configuration files using terraform validate.

Generate an execution plan using terraform plan.

Deploy Azure resources using terraform apply.

Infrastructure automation also simplifies disaster recovery and scalability. If infrastructure failure occurs, the entire cloud environment can be recreated quickly using the Terraform configuration files. This reduces manual effort, deployment errors, and infrastructure management complexity.

Terraform is integrated with GitHub Actions in the CI/CD pipeline to automate infrastructure provisioning during deployment. This combination enables continuous deployment, faster updates, secure cloud management, and efficient resource utilization within the PETRA platform.

3.3 CONTAINERIZATION STRATERGY (DOCKER)

Docker is used in the PETRA system to containerize backend services, APIs, and supporting application components. Containerization helps package the application along with all its dependencies, libraries, and runtime environments into lightweight and portable containers. This ensures that the application behaves consistently across development, testing, and production environments.

The backend services developed using Flask APIs are deployed inside Docker containers. These containers provide isolated execution environments, reducing dependency conflicts and simplifying deployment processes. Docker images are created using Dockerfiles that define the application runtime, dependencies, configurations, and execution commands.

The containerization strategy in PETRA provides several advantages:

Simplified deployment and management of applications.

Consistent runtime environment across all platforms.

Faster application scaling and resource allocation.

Improved portability between local systems and cloud environments.

Reduced dependency and compatibility issues.

Easier integration with cloud-native deployment workflows.

Docker containers are integrated with Azure cloud services and deployed using automated CI/CD pipelines implemented through GitHub Actions. Whenever changes are pushed to the repository, the CI/CD workflow automatically builds Docker images, performs testing, and deploys updated containers to the Azure environment.

The platform also supports scalability through container orchestration and cloud auto-scaling features. This enables PETRA to efficiently handle increasing user requests, AI processing workloads, and background tasks while maintaining application reliability and performance.

```
# Multi-stage build with scratch for minimal size
FROM golang:1.23-alpine AS builder

# Install git for go mod download
RUN apk add --no-cache git ca-certificates tzdata

WORKDIR /app

# Copy go mod files first for better caching
COPY go.mod go.sum ./
RUN go mod download

# Copy source code
COPY . .

# Build static binary
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -ldflags '-w -s' -o marketplace-server
main.go

# Final stage - scratch (empty image)
FROM scratch

# Copy certificates and timezone data from builder
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo

# Copy binary from builder stage
COPY --from=builder /app/marketplace-server /marketplace-server

# Expose port
EXPOSE 8080

# Run the binary
ENTRYPOINT ["/marketplace-server"]
```

Figure 4. Docker file

3.4 KUBERNETES ORCHESTRATION

Azure Kubernetes Service (AKS) is utilised for container orchestration, gain control over scaling, security, and traffic.

1. Cluster and Deployment Design

Cluster Structure: The cluster uses a multi-pool architecture (System and User) with Azure CNI networking. It includes the Cluster Autoscaler to dynamically manage node capacity based on demand.

Workload: The Go backend is deployed as a Deployment within a dedicated namespace (e.g., marketplace-app). It relies on standard Kubernetes liveness and readiness probes targeting the / health endpoint.

Ingress: Traffic is managed by an Ingress Controller (e.g., NGINX), which handles TLS termination and strictly enforces HTTPS-only external access.

Configuration: Non-sensitive settings use ConfigMaps. Sensitive data is managed through Azure Key Vault and securely injected into the pods via the Secrets Store CSI Driver.

2. Scaling, Resilience, and Security

Pod Scaling: The Horizontal Pod Autoscaler (HPA) automatically scales the number of replicas based on resource metrics like CPU Utilization (e.g., 60%).

Node Scaling: The Cluster Autoscaler adjusts the underlying node count to meet the HPA's capacity requirements.

Resilience: Rolling Updates are configured with settings like maxSurge:25% and maxUnavailable:0 for zero-downtime deployments.

3.5 GENAI INTEGRATION AND AZURE AI SERVICE MAPPING

The PETRA system integrates Generative AI (GenAI) and Azure AI services to automate petition classification, intelligent processing, and priority-based handling. These AI services improve operational efficiency by reducing manual work, enabling faster decision-making, and ensuring that urgent petitions are processed quickly.

The platform uses Natural Language Processing (NLP) models such as Scikit-learn, NLTK, and spaCy to analyze petition content and categorize requests based on their type, urgency, and keywords. The AI module processes petition text, identifies important information, and assigns priority scores to ensure that critical issues receive immediate attention.

Azure Functions are used as the primary serverless processing layer for AI operations. When a user submits a petition, Azure Functions automatically trigger the AI workflow to perform classification, priority evaluation, and notification generation. This event-driven processing architecture enables scalable and efficient handling of large numbers of petitions.

The GenAI integration in PETRA provides the following functionalities:

1. Automatic petition categorization using NLP models.
2. Rule-based priority scoring for urgent requests.
3. Intelligent text analysis and keyword extraction.
4. Automated processing and workflow management.
5. Real-time notification generation for users and administrators.
6. Background processing for scheduled tasks and reminders.
7. Azure AI-related services used in the platform include:
8. Azure Functions – Executes AI-based classification and serverless processing tasks.
9. Azure Application Insights – Monitors AI processing performance and logs.
10. Azure SQL Database – Stores classified petition data and processing results.
11. Power BI Embedded – Visualizes analytics and petition trends generated from AI processing.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

4.1 DEVSECOPS INTEGRATION

Security is integrated throughout the complete software development lifecycle in the PETRA system using DevSecOps practices. The project follows a secure-by-design approach where security measures are incorporated during development, testing, deployment, and monitoring stages to ensure data protection, secure access, and reliable cloud operations.

The platform uses HTTPS encrypted communication to secure data transfer between users, frontend applications, backend APIs, and cloud services. This encryption protects sensitive user information, petition details, and uploaded documents from unauthorized access during transmission.

Sensitive configuration details such as database credentials, API keys, authentication tokens, and cloud service secrets are managed using secure environment variables instead of hardcoding them into the application source code. GitHub Secrets are integrated within the CI/CD pipeline to securely manage deployment credentials and cloud authentication details during automated workflows.

Authentication and identity management are implemented using Azure AD B2C, which provides secure login functionality, role-based access control, and user identity protection. JWT-based authentication mechanisms are used to validate users and secure API access across the platform.

The system also implements logging and audit tracking to monitor application activities, user actions, API requests, and system events. Azure Application Insights is used for centralized monitoring, error tracking, and security-related log analysis. These logs help identify suspicious activities, troubleshoot issues, and maintain operational transparency.

Docker container security practices are followed to ensure secure application deployment. The project uses containerized backend services with controlled dependencies, isolated runtime environments, and secure deployment configurations. Automated CI/CD workflows validate deployments before releasing updates to production environments.

By integrating DevSecOps principles, PETRA ensures secure software delivery, continuous monitoring, reduced security vulnerabilities, and reliable cloud-native operations throughout the application lifecycle.

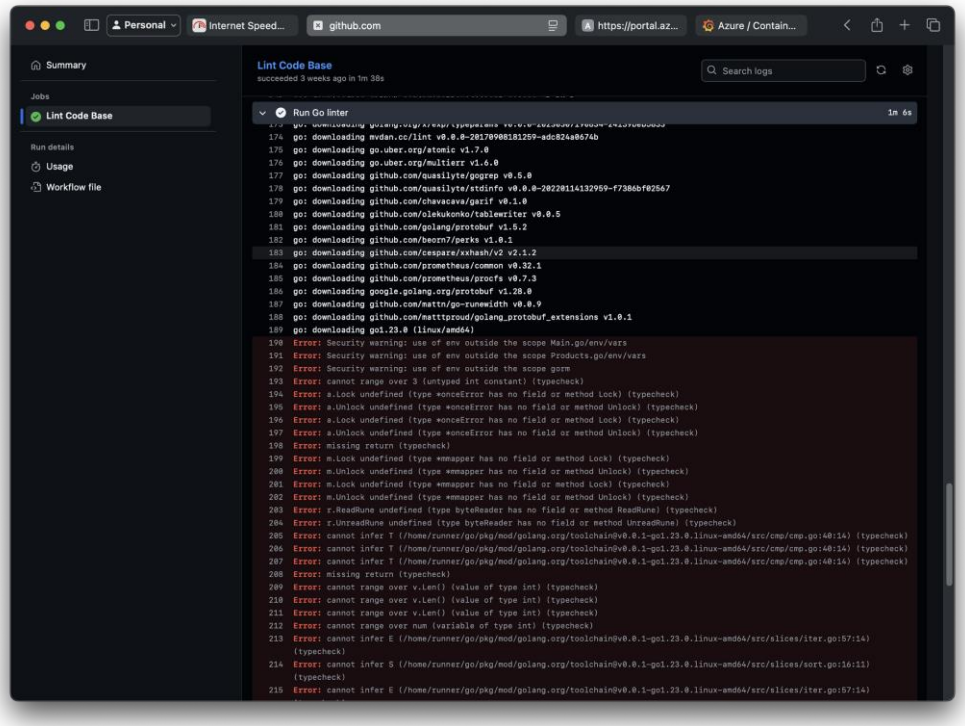


Figure 5. Github Workflow Showing Lint Errors

After: Errors resolved

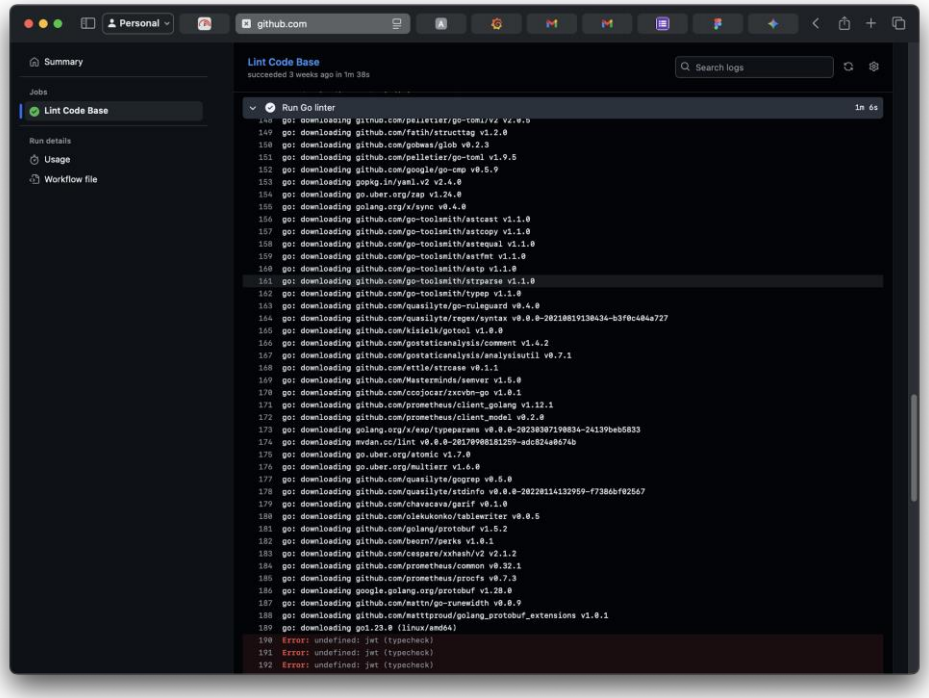


Figure 6. Github Workflow Showing Resolved Env Errors

4.2MONITORING AND OBSERVABILITY

The PETRA system implements comprehensive monitoring and observability mechanisms to ensure high performance, reliability, and continuous system availability. Monitoring services help track application health, system performance, user activities, and cloud resource utilization in real time. Observability tools provide insights into system behavior, helping administrators quickly identify and resolve issues.

Azure Application Insights is integrated into the platform to monitor frontend and backend application performance. It collects telemetry data such as request rates, response times, API failures, exceptions, and dependency performance. This enables developers and administrators to analyze application behavior and improve system efficiency.

Azure Monitor is used to track cloud infrastructure performance, resource utilization, and operational metrics. It continuously monitors CPU usage, memory consumption, network activity, and service health across Azure App Service, Azure Functions, and database resources. Alerts and notifications are automatically generated when predefined performance thresholds are exceeded.

The platform also uses centralized logging and audit tracking to maintain operational transparency and security monitoring. Logs generated from APIs, authentication systems, background jobs, and AI processing modules are collected and analyzed for troubleshooting and system optimization.

Power BI Embedded provides analytics dashboards and visual reports that display petition statistics, processing trends, user activities, priority distributions, and system performance metrics. These dashboards support data-driven decision-making and administrative monitoring.

- Monitoring and observability in PETRA provide several benefits including:
- Real-time performance tracking.
- Error detection and troubleshooting.
- Resource utilization monitoring.
- Automated alert generation.
- Improved operational efficiency.
- Faster incident response.
- Better system reliability and availability.

By integrating Azure monitoring tools and analytics services, the PETRA platform ensures continuous visibility into system operations, enabling proactive maintenance, efficient performance management, and reliable cloud-based petition processing.

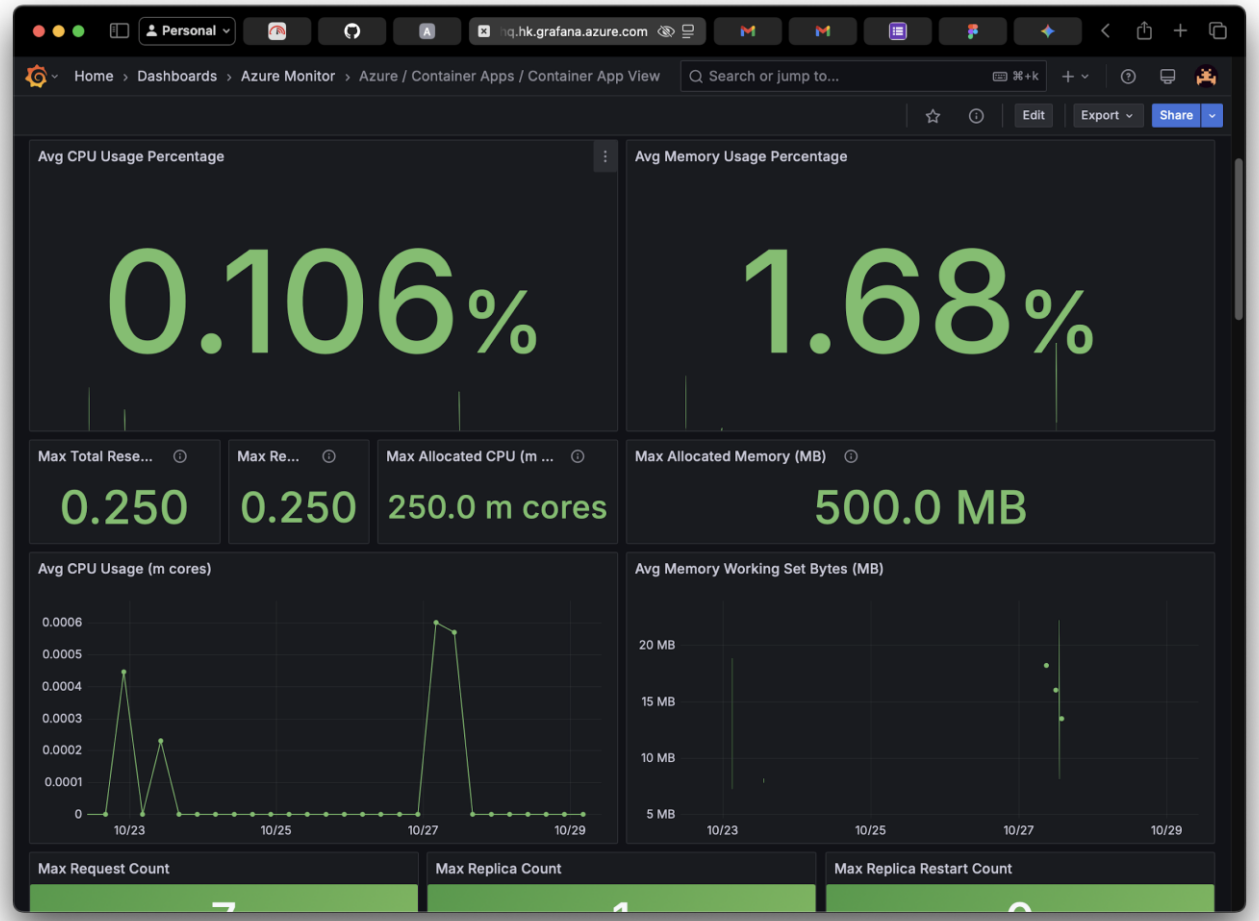


Figure 6. Grafana Dashboard

4.3 ACCESS CONTROL

The PETRA system implements secure access control mechanisms to ensure that only authorized users can access specific functionalities and data within the platform. Access control is managed using role-based authentication and authorization techniques, providing secure and controlled access to petitions, administrative tools, and cloud resources.

Azure AD B2C is integrated into the system for secure user authentication and identity management. Users are required to log in using verified credentials before accessing the platform. The authentication process uses secure HTTPS communication and token-based validation to protect user sessions and sensitive information.

The platform follows Role-Based Access Control (RBAC), where different access permissions are assigned based on user roles. The primary user roles include:

- **User/Student** – Can submit petitions, upload documents, track request status, and receive notifications.
- **Administrator** – Can review, classify, prioritize, approve, or reject petitions and manage users.
- **Department Authority** – Can monitor and process petitions related to specific departments or categories.
- **Monitoring Team** – Can access analytics dashboards and performance reports.

JWT (JSON Web Token) authentication is used to validate user sessions and secure API communication between the frontend and backend services. Tokens are verified before granting access to protected endpoints and resources.

Sensitive information such as database credentials, API keys, and authentication secrets are securely stored using environment variables and GitHub Secrets within the CI/CD pipeline. Access to cloud resources is restricted through Azure security policies and firewall configurations.

The access control mechanism in PETRA provides:

- Secure user authentication.
- Controlled access based on user roles.
- Protection of sensitive petition data.
- Secure API and cloud resource access.

Improved system security and accountability.

By implementing Azure AD B2C, RBAC, and JWT-based authentication, the PETRA platform ensures secure and reliable access management throughout the application.

4.4 BLUE–GREEN DEPLOYMENT & DISASTER RECOVERY PLANNING

The PETRA system follows a Blue–Green deployment strategy to ensure zero downtime, safe application updates, and reliable cloud operations. In this deployment model, two identical production environments are maintained: the Blue environment, which represents the currently active application version, and the Green environment, which contains the updated version of the application.

When a new update is developed, it is first deployed and tested in the Green environment without affecting active users. After successful testing and validation, traffic is gradually redirected from the Blue environment to the Green environment. This approach minimizes deployment risks, reduces service interruption, and enables quick rollback in case of failures or unexpected issues.

The deployment process is automated using GitHub Actions and Docker-based CI/CD pipelines. Docker containers ensure consistent application environments, while Azure App Service and Azure Functions support scalable and reliable cloud deployment. Health checks and monitoring services validate application performance before switching production traffic.

The disaster recovery planning in PETRA is designed to ensure data protection, business continuity, and rapid system restoration during failures or outages. Azure cloud services provide built-in backup and redundancy mechanisms to maintain system reliability.

Key disaster recovery strategies include:

- Automated backups of Azure SQL Database.
- Redundant storage using Azure Blob Storage replication.
- Infrastructure recreation using Terraform Infrastructure-as-Code.
- Version-controlled application deployments through GitHub.
- Continuous monitoring using Azure Application Insights.
- Rollback support through Blue–Green deployment architecture.

In the event of infrastructure failure, database corruption, or deployment issues, the system can quickly restore services using backup data and Terraform deployment configurations. This reduces downtime, prevents data loss, and ensures uninterrupted petition management services.

The combination of Blue–Green deployment and disaster recovery planning improves system availability, operational stability, deployment reliability, and overall cloud resilience within the PETRA platform.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The PETRA – Petition Management System was successfully designed, developed, and deployed as a cloud-based AI-powered platform using Microsoft Azure services. The system effectively automates petition submission, intelligent classification, tracking, monitoring, and administrative processing through a centralized digital platform.

The frontend application was developed using React and Flutter to provide a responsive and user-friendly interface for petition submission, tracking, and notifications. Backend services were implemented using Flask APIs to manage authentication, petition workflows, status updates, and communication between cloud services.

Azure App Service was used for hosting the frontend and backend applications, ensuring scalable and reliable deployment. Azure Functions were integrated for serverless processing of AI-based petition classification, automated notifications, scheduled reminders, and background tasks. Azure SQL Database was implemented for storing structured data such as user details, petition records, logs, and processing results, while Azure Blob Storage securely handled PDF and audio file uploads.

The platform also integrated AI and NLP technologies such as Scikit-learn, NLTK, and spaCy for intelligent petition categorization and priority scoring. These AI models enabled automatic classification of petitions and ensured that urgent requests were processed more efficiently.

Secure authentication and identity management were implemented using Azure AD B2C with role-based access control and JWT-based authorization mechanisms. Monitoring and observability were achieved using Azure Application Insights and Power BI Embedded dashboards, which provided real-time analytics, performance tracking, and reporting insights.

The deployment process was automated using Docker, Terraform, and GitHub Actions, enabling continuous integration and continuous deployment (CI/CD). The cloud-native architecture ensured scalability, fault tolerance, high availability, and operational efficiency.

Overall, the implementation of PETRA successfully achieved the project objectives by providing a secure, intelligent, scalable, and efficient petition management platform that improves transparency, reduces manual effort, and enhances user satisfaction through cloud computing and AI integration.

5.2 CHALLENGES FACED AND RESOLUTIONS

During the development and deployment of the PETRA – Petition Management System, several technical and operational challenges were encountered. These challenges were addressed through appropriate cloud technologies, AI integration techniques, and infrastructure optimization strategies.

Challenge 1 – Managing Large File Uploads

The system supports uploading PDF documents and audio files along with petitions. Handling large media files initially caused increased storage usage and slower processing performance.

Resolution:
Azure Blob Storage was integrated to provide scalable and secure cloud storage for uploaded files. This reduced database load, improved file management efficiency, and enabled faster access to documents and audio content.

Challenge 2 – AI Classification Accuracy

The initial NLP-based petition classification system produced inconsistent categorization results for certain petition types due to variations in user-written text.

Resolution:
Additional preprocessing techniques, keyword extraction, and rule-based priority scoring mechanisms were implemented using Scikit-learn, NLTK, and spaCy models. This improved classification accuracy and enhanced priority-based handling of urgent petitions.

Challenge 3 – Real-Time Notification Delays

Notification processing experienced delays during high workloads because background tasks and alert services were not efficiently managed initially.

Resolution:
Azure Functions with event-driven triggers and queue-based background processing were implemented to handle notifications, reminders, and scheduled tasks efficiently. This significantly improved real-time alert delivery and system responsiveness.

Challenge 4 – Secure Authentication and Access Management

Managing secure authentication, user sessions, and role-based access control was a critical challenge due to the need for protected petition data and secure administrative operations.

Resolution:

Azure AD B2C was integrated for secure authentication and identity management. JWT-based authorization and HTTPS communication were implemented to ensure protected access to APIs and user data.

Challenge 5 – Deployment and Infrastructure Management

Manual deployment and cloud resource configuration initially resulted in inconsistent environments and increased maintenance complexity.

Resolution:

Terraform Infrastructure-as-Code (IaC), Docker containerization, and GitHub Actions CI/CD pipelines were implemented to automate deployment, infrastructure provisioning, and cloud resource management. This improved deployment consistency, scalability, and operational reliability.

Challenge 6 – Monitoring and System Reliability

Tracking application performance, identifying errors, and maintaining system reliability across multiple cloud services became difficult during testing and scaling phases.

Resolution:

Azure Application Insights and Azure Monitor were integrated to provide real-time monitoring, logging, performance analysis, and alert generation. This enabled proactive issue detection and improved operational stability.

By addressing these challenges with cloud-native solutions and DevOps practices, the PETRA platform achieved secure deployment, scalable infrastructure, intelligent automation, and reliable petition management operations.

5.3 PERFORMANCE OR COST OBSERVATION

Performance Benchmarking

The PETRA system demonstrated reliable performance, scalability, and operational efficiency during testing and deployment on Microsoft Azure cloud infrastructure. The cloud-native architecture and serverless processing model enabled the platform to handle petition management operations efficiently while maintaining low response times and high availability.

The frontend application hosted on Azure App Service provided fast and responsive user interactions for petition submission, tracking, and dashboard access. Average application response time remained below 2 seconds under normal workloads, ensuring a smooth user experience.

Backend APIs developed using Flask efficiently processed petition requests, authentication, AI classification, and database operations with minimal latency.

Azure Functions played a significant role in improving performance through event-driven serverless execution. AI-based petition classification, notifications, scheduled reminders, and background processing tasks were automatically scaled based on workload demands. This reduced processing delays and improved real-time responsiveness for users and administrators.

The integration of NLP models such as Scikit-learn, NLTK, and spaCy successfully automated petition categorization and priority scoring. AI processing workflows effectively identified urgent petitions and reduced manual administrative effort. Power BI Embedded dashboards provided real-time analytics and reporting insights without affecting core application performance.

Monitoring tools such as Azure Application Insights and Azure Monitor continuously tracked CPU usage, request rates, response times, exceptions, and cloud resource utilization. Auto-scaling capabilities in Azure App Service and Azure Functions allowed the platform to efficiently support increasing workloads and concurrent users.

Cost Analysis and Optimization

The PETRA platform was designed with cost optimization strategies to minimize operational expenses while maintaining scalability and reliability. Azure App Service provided cost-effective hosting for frontend and backend applications, while Azure Functions reduced infrastructure costs through pay-as-you-use serverless execution.

Azure Blob Storage offered scalable and low-cost storage for PDFs and audio files, preventing unnecessary database storage overhead. Auto-scaling cloud services optimized resource utilization by allocating compute resources only when required. Docker containerization and automated CI/CD pipelines further improved deployment efficiency and reduced maintenance costs.

The estimated monthly operational cost varied depending on workload and usage levels, with major cost contributors including Azure App Service, Azure SQL Database, Power BI Embedded, and monitoring services. Despite these expenses, the use of cloud-native services, serverless processing, and scalable infrastructure ensured that the platform remained cost-efficient and suitable for handling large-scale petition management operations.

5.3 PERFORMANCE OR COST OBSERVATIONS

The PETRA system demonstrated efficient performance, scalability, and cost optimization during deployment and testing on Microsoft Azure cloud infrastructure. The cloud-native architecture enabled the platform to process petitions, notifications, AI classification, and analytics efficiently while maintaining reliable system availability and fast response times.

The frontend application hosted on Azure App Service provided responsive petition submission and tracking with an average response time below 2 seconds. Backend Flask APIs efficiently handled authentication, petition workflows, and database communication with minimal processing delays. Azure Functions improved system performance by enabling event-driven execution for AI-based classification, background tasks, and automated notifications.

The integration of NLP models such as Scikit-learn, NLTK, and spaCy successfully automated petition categorization and priority scoring, reducing manual administrative effort and improving processing efficiency. Power BI Embedded dashboards generated real-time analytics and monitoring insights without affecting application performance.

From a cost perspective, the system was designed using scalable and cost-efficient Azure services. Azure Functions reduced infrastructure expenses through serverless execution, while Azure Blob Storage minimized storage costs for PDFs and audio files. Auto-scaling mechanisms in Azure App Service and Azure Functions optimized resource usage by dynamically allocating compute resources based on workload demands.

Monitoring tools such as Azure Application Insights and Azure Monitor helped track resource utilization, request rates, errors, and operational costs in real time. The major operational costs were associated with Azure App Service, Azure SQL Database, Power BI Embedded, and monitoring services. However, the use of cloud-native services, Infrastructure-as-Code, Docker containerization, and CI/CD automation significantly reduced maintenance overhead and deployment complexity.

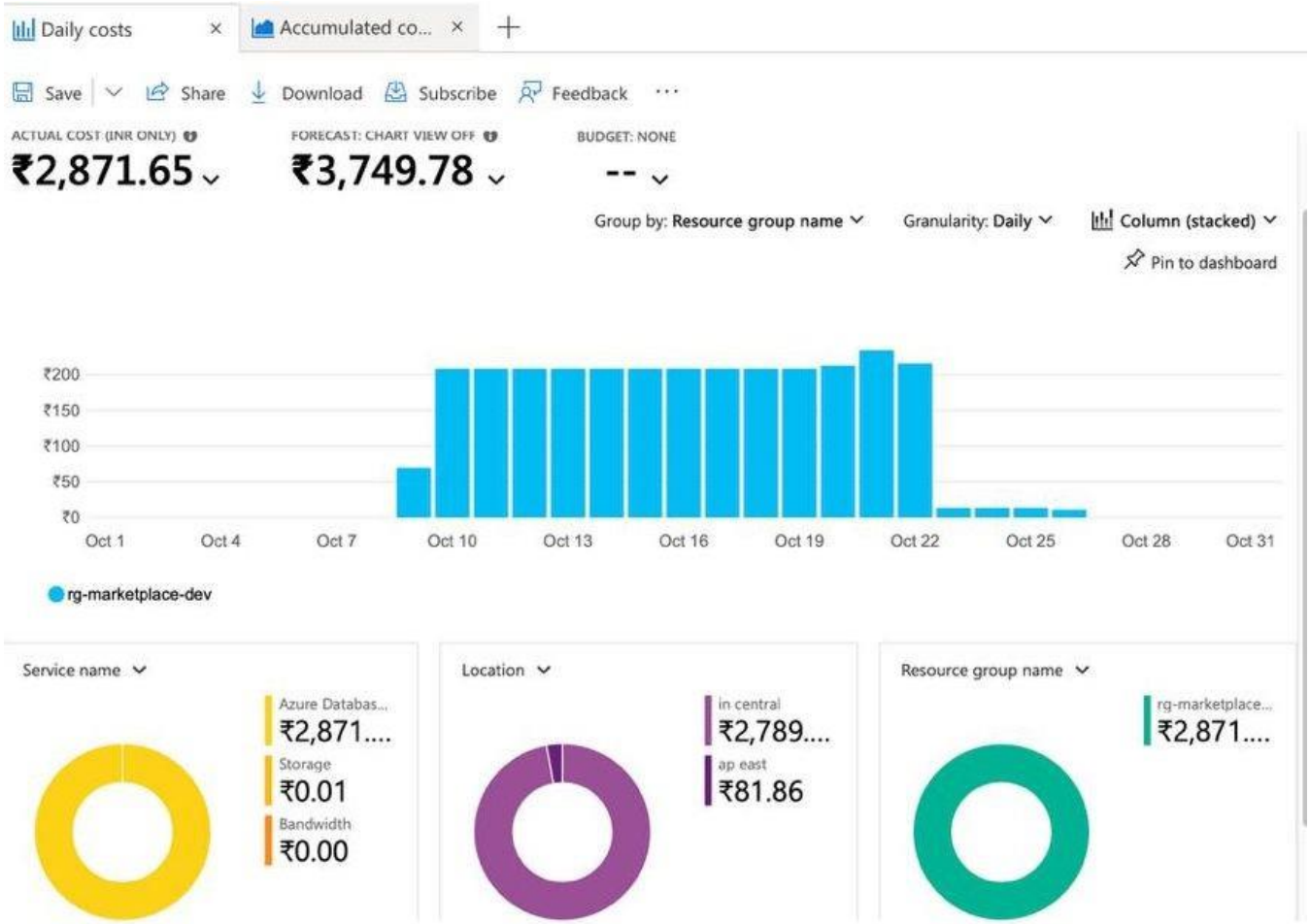


Figure 7. Daily Cost Grap

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of the PETRA – Petition Management System provided valuable practical experience in cloud computing, AI integration, DevOps practices, and scalable application deployment. The project helped the team understand the complete lifecycle of designing, developing, deploying, and monitoring a cloud-native application using Microsoft Azure services.

One of the major learnings from the project was the implementation of cloud-based infrastructure using Azure App Service, Azure Functions, Azure SQL Database, and Azure Blob Storage. The team gained hands-on experience in deploying scalable applications, configuring cloud services, and managing secure cloud environments.

The project also provided significant exposure to Artificial Intelligence and Natural Language Processing technologies. By integrating NLP models such as Scikit-learn, NLTK, and spaCy, the team learned how AI can be used for intelligent petition classification, priority scoring, and

automated processing workflows. This improved understanding of machine learning concepts and AI-based automation techniques.

Another important learning area was DevOps and Infrastructure-as-Code practices. The team implemented Docker containerization, GitHub Actions CI/CD pipelines, and Terraform-based infrastructure provisioning. These technologies helped automate deployment processes, improve infrastructure consistency, and simplify cloud resource management.

The integration of Azure Application Insights and Power BI Embedded provided practical experience in monitoring, observability, analytics, and performance optimization. The team learned how real-time dashboards and monitoring tools support operational efficiency and data-driven decision-making.

The project also strengthened teamwork, communication, and collaborative development skills. Tasks were divided among team members based on frontend development, backend APIs, cloud deployment, AI processing, database management, and analytics integration. Regular discussions, testing, debugging sessions, and version control practices ensured smooth project progress and efficient coordination among team members.

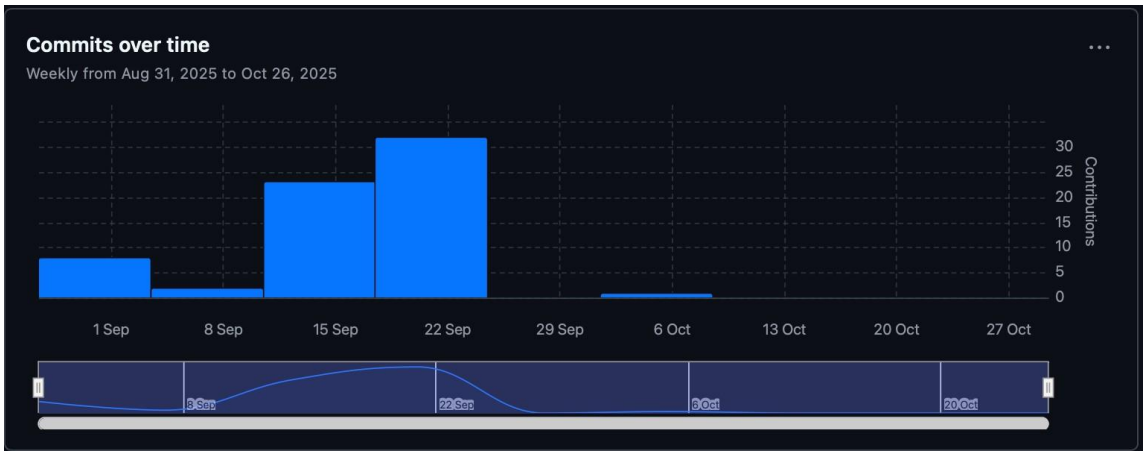


Figure 8. Github Contributions

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The PETRA – Petition Management System successfully modernizes traditional petition and complaint handling processes through cloud computing, AI integration, and automated workflow management. The system provides a centralized, secure, and scalable platform that enables users to submit petitions with supporting documents, track request status in real time, and receive automated notifications efficiently.

By integrating Microsoft Azure cloud services such as Azure App Service, Azure Functions, Azure SQL Database, Azure Blob Storage, Azure AD B2C, and Power BI Embedded, the platform achieves high availability, scalability, reliability, and operational efficiency. The use of AI and NLP models including Scikit-learn, NLTK, and spaCy enables intelligent petition classification and priority-based processing, ensuring that urgent requests are handled quickly and effectively.

The implementation of DevOps practices such as Docker containerization, Terraform Infrastructure-as-Code, and GitHub Actions CI/CD pipelines improved deployment automation, infrastructure consistency, and application maintainability. Monitoring and observability tools such as Azure Application Insights further enhanced system reliability and performance management.

Overall, the PETRA platform successfully addresses the limitations of manual petition management systems by improving transparency, reducing administrative workload, enabling data-driven decision-making, and enhancing user satisfaction through intelligent automation and cloud-native deployment. The project demonstrates the effective application of cloud computing, AI technologies, DevOps practices, and scalable system architecture in developing a modern petition management solution.

6.2 FUTURE WORKS

The PETRA – Petition Management System provides a strong foundation for future enhancements and advanced intelligent features. Future developments of the platform will focus on improving automation, user experience, scalability, and advanced analytics capabilities.

One of the major future enhancements includes the development of a dedicated mobile application for Android and iOS platforms to provide easier access for users to submit and track petitions from mobile devices. The platform can also be extended with multilingual support to allow users to interact with the system in multiple regional languages.

Advanced AI capabilities can be integrated to improve petition analysis and decision-making. Future AI enhancements may include sentiment analysis, predictive analytics, and deep learning models to better identify urgent petitions, detect fraudulent requests, and forecast petition trends. Voice-to-text processing can also be implemented to convert audio complaints directly into text for automated classification.

The system can further integrate chatbot-based assistance using Generative AI technologies to guide users during petition submission, answer common queries, and provide instant support. Real-time conversational interfaces can improve accessibility and user engagement.

Future versions of PETRA can also support integration with ERP systems, government portals, and institutional management systems for seamless data exchange and centralized administration. Blockchain-based audit tracking may be introduced to improve transparency, security, and tamper-proof record management.

Additional monitoring and analytics capabilities can be implemented using advanced Power BI dashboards and AI-driven reporting systems to provide deeper operational insights and improve decision-making processes.

APPENDICES

Appendix A – Terraform Code Snippets

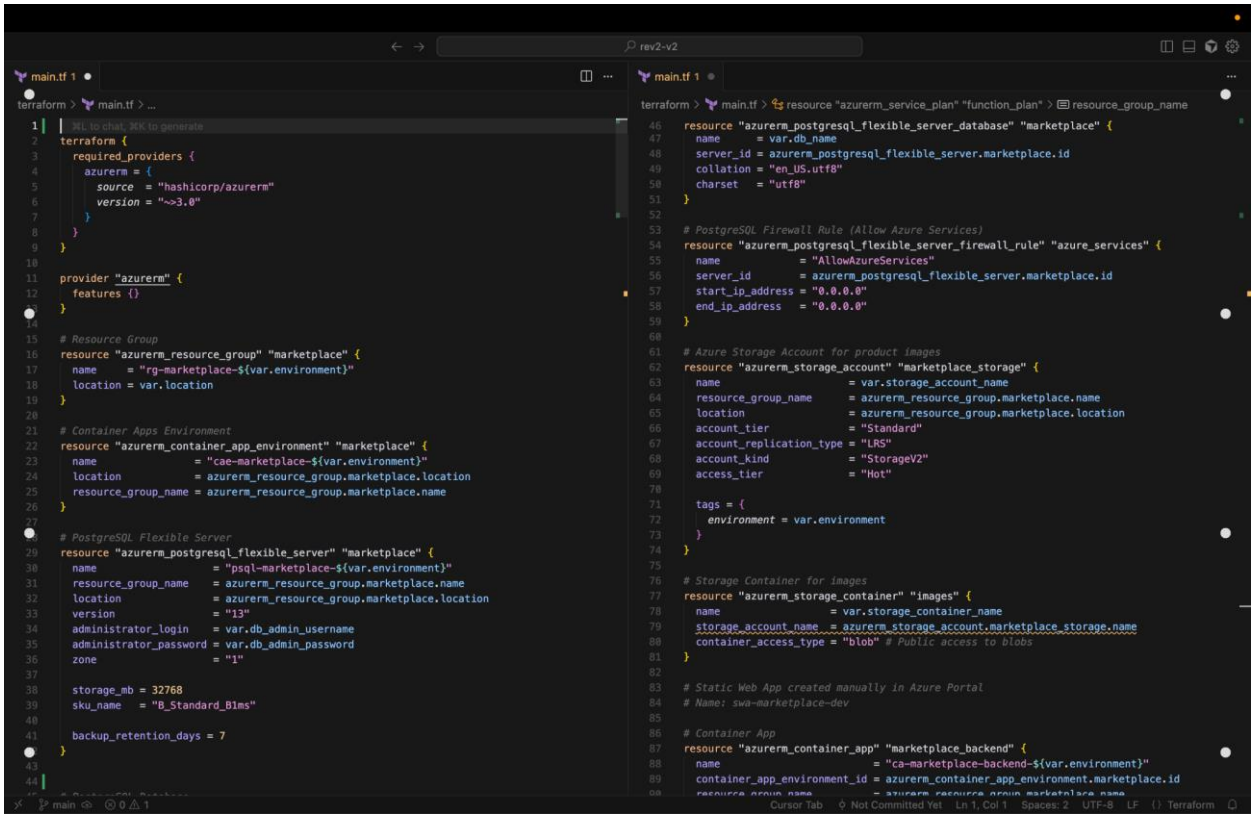
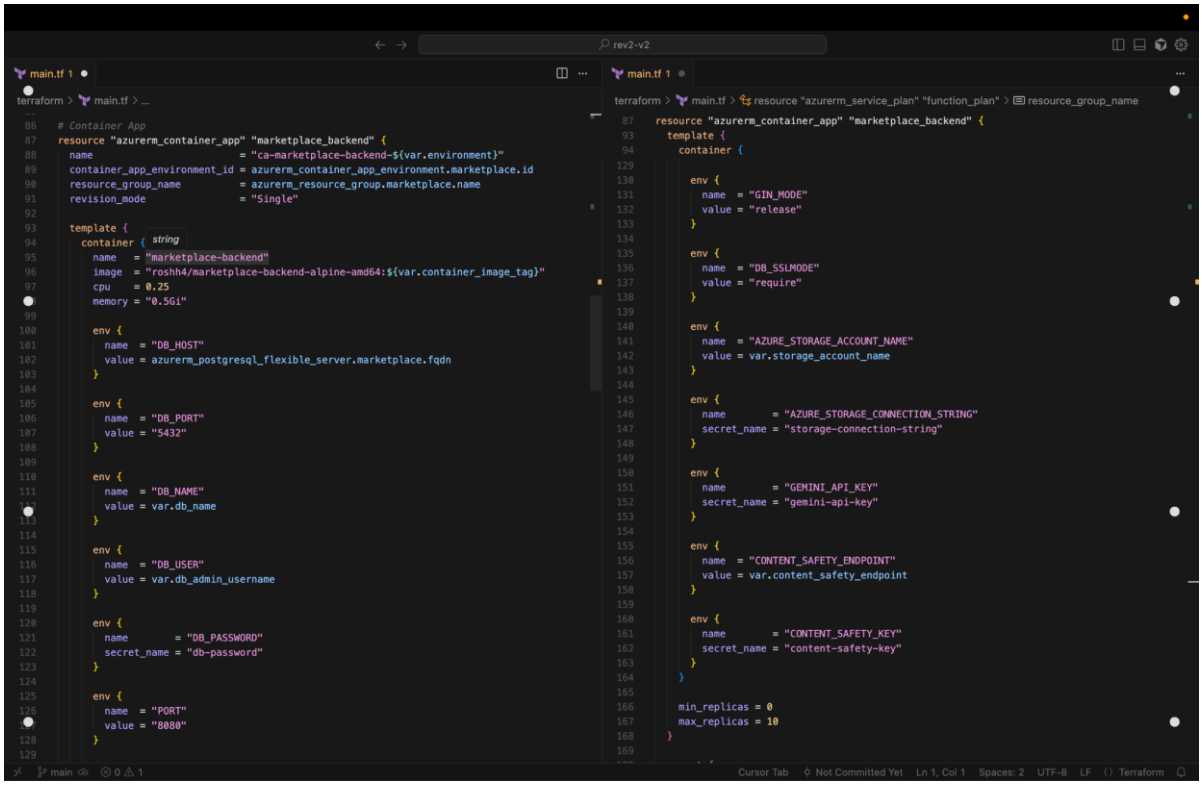


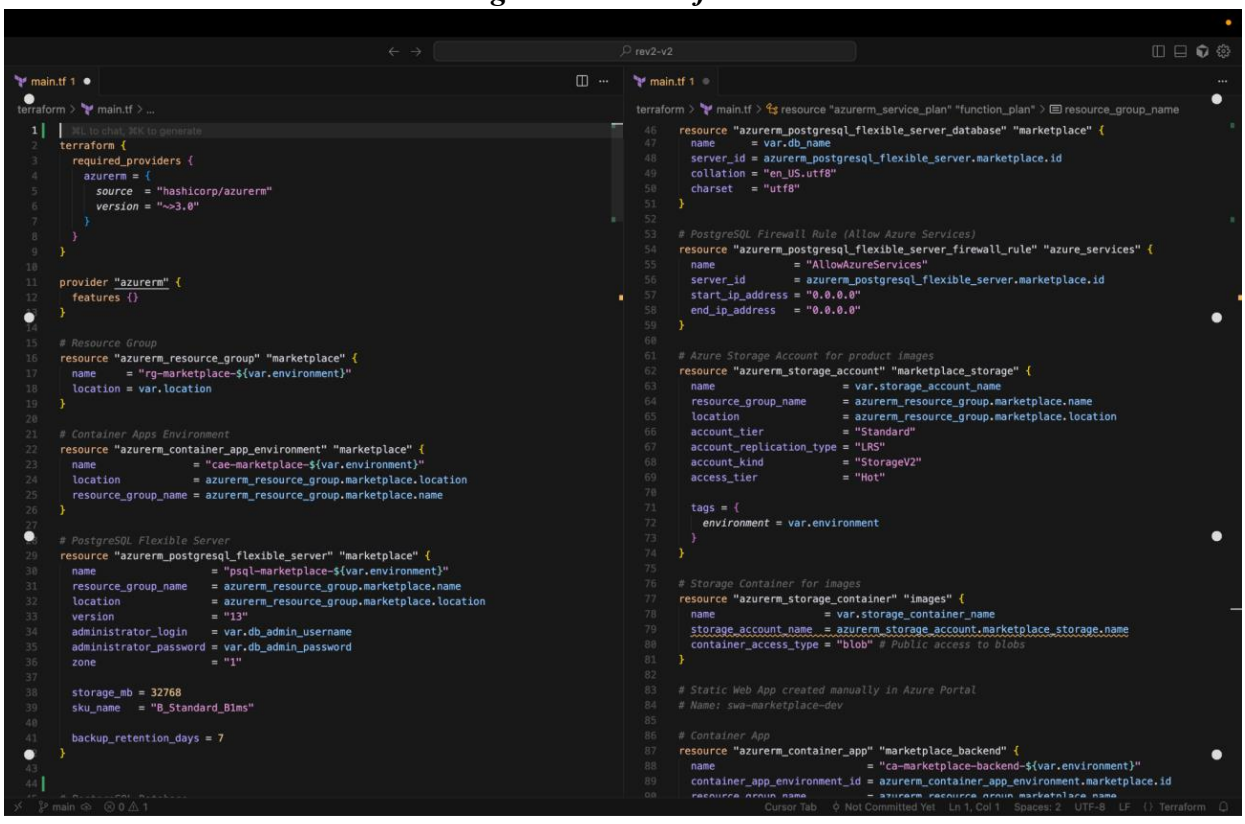
Figure 9. Main.tf code



```
# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
  resource_group_name          = azurerm_resource_group.marketplace.name
  revision_mode                = "Single"

  template {
    container {
      name   = "marketplace-backend"
      image  = "roshh4/marketplace-backend-alpine-amd64:${var.container_image_tag}"
      cpu    = 0.25
      memory = "0.5Gi"

      env {
        name  = "DB_HOST"
        value = azurerm_postgresql_flexible_server.marketplace.fqdn
      }

      env {
        name  = "DB_PORT"
        value = "5432"
      }

      env {
        name  = "DB_NAME"
        value = var.db_name
      }

      env {
        name  = "DB_USER"
        value = var.db_admin_username
      }

      env {
        name        = "DB_PASSWORD"
        secret_name = "db-password"
      }

      env {
        name  = "PORT"
        value = "8080"
      }
```

```
resource "azurerm_container_app" "marketplace_backend" {
  template {
    container {

      env {
        name  = "GIN_MODE"
        value = "release"
      }

      env {
        name  = "DB_SSLMODE"
        value = "require"
      }

      env {
        name  = "AZURE_STORAGE_ACCOUNT_NAME"
        value = var.storage_account_name
      }

      env {
        name        = "AZURE_STORAGE_CONNECTION_STRING"
        secret_name = "storage-connection-string"
      }

      env {
        name        = "GEMINI_API_KEY"
        secret_name = "gemini-api-key"
      }

      env {
        name  = "CONTENT_SAFETY_ENDPOINT"
        value = var.content_safety_endpoint
      }

      env {
        name        = "CONTENT_SAFETY_KEY"
        secret_name = "content-safety-key"
      }
    }

    min_replicas = 0
    max_replicas = 10
  }
```

Figure 10. Main.tf code

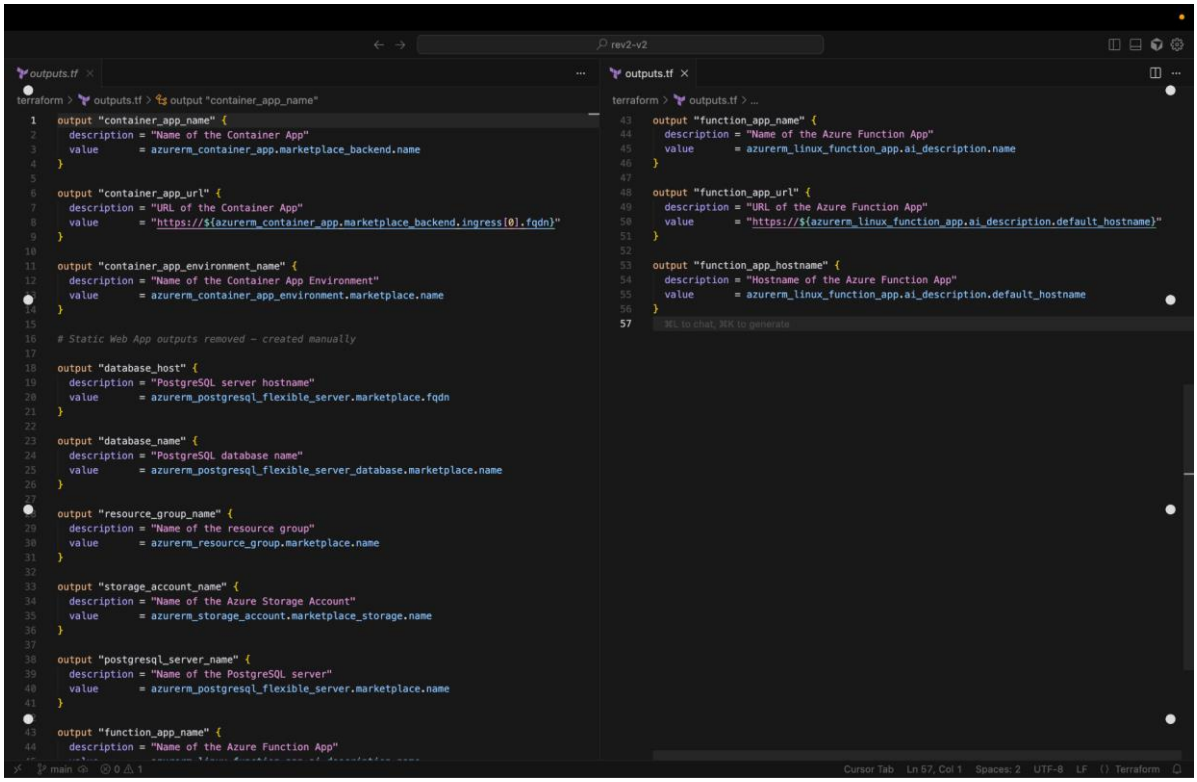


```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# Resource Group
resource "azurerm_resource_group" "marketplace" {
  name     = "rg-marketplace-${var.environment}"
  location = var.location
}

# Container Apps Environment
resource "azurerm_container_app_environment" "marketplace" {
  name                = "cae-marketplace-${var.environment}"
  location            = azurerm_resource_group.marketplace.location
  resource_group_name = azurerm_resource_group.marketplace.name
}

# PostgreSQL Flexible Server
resource "azurerm_postgresql_flexible_server" "marketplace" {
  name                   = "psql-marketplace-${var.environment}"
  resource_group_name    = azurerm_resource_group.marketplace.name
  location               = azurerm_resource_group.marketplace.location
  version                = "13"
  administrator_login    = var.db_admin_username
  administrator_password = var.db_admin_password
  zone                   = "1"

  storage_mb = 32768
  sku_name   = "B_Standard_B1ms"

  backup_retention_days = 7
}
```

```
resource "azurerm_postgresql_flexible_server_database" "marketplace" {
  name      = var.db_name
  server_id = azurerm_postgresql_flexible_server.marketplace.id
  collation = "en_US.utf8"
  charset   = "utf8"
}

# PostgreSQL Firewall Rule (Allow Azure Services)
resource "azurerm_postgresql_flexible_server_firewall_rule" "azure_services" {
  name             = "AllowAzureServices"
  server_id        = azurerm_postgresql_flexible_server.marketplace.id
  start_ip_address = "0.0.0.0"
  end_ip_address   = "0.0.0.0"
}

# Azure Storage Account for product images
resource "azurerm_storage_account" "marketplace_storage" {
  name                     = var.storage_account_name
  resource_group_name      = azurerm_resource_group.marketplace.name
  location                 = azurerm_resource_group.marketplace.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  access_tier              = "Hot"

  tags = {
    environment = var.environment
  }
}

# Storage Container for images
resource "azurerm_storage_container" "images" {
  name                  = var.storage_container_name
  storage_account_name  = azurerm_storage_account.marketplace_storage.name
  container_access_type = "blob" # Public access to blobs
}

# Static Web App created manually in Azure Portal
# Name: swa-marketplace-dev

# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
```

Figure 11. Main.tf code

outputs.tf

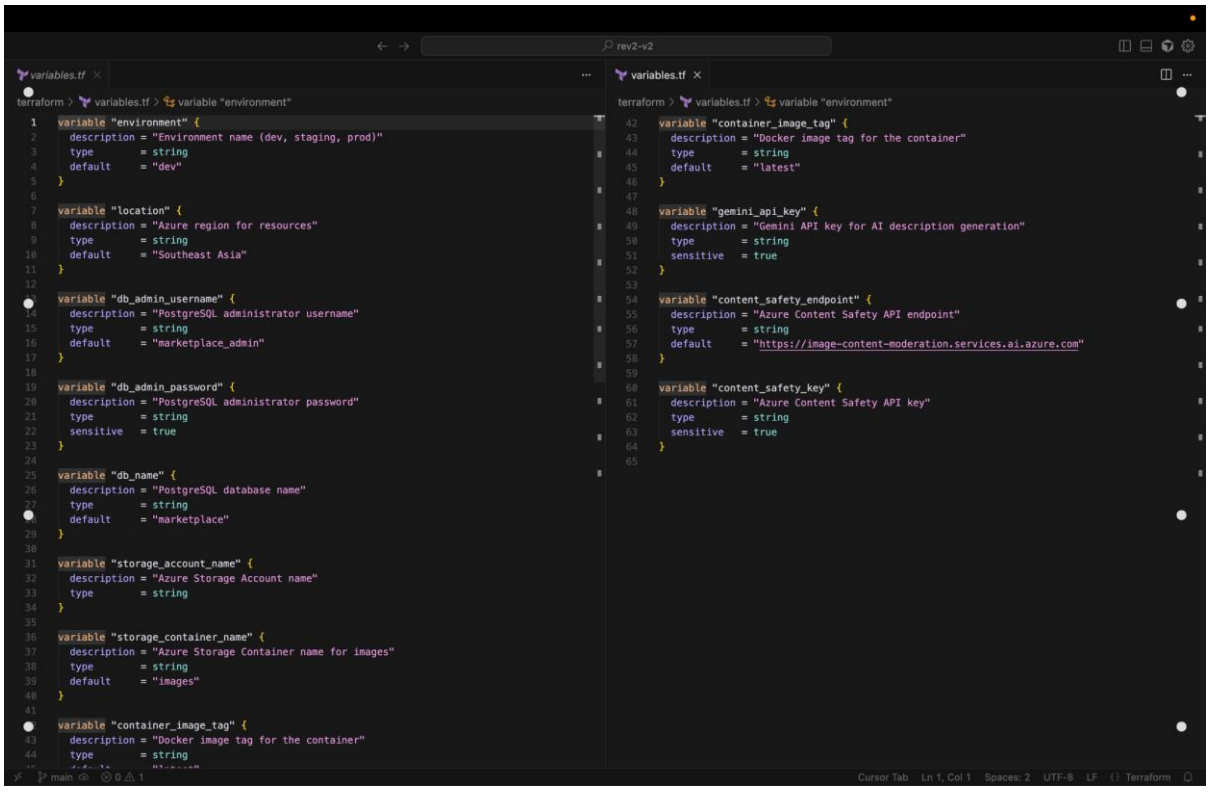


```
output "container_app_name" {
  description = "Name of the Container App"
  value       = azurerm_container_app.marketplace_backend.name
}

output "container_app_url" {
  description = "URL of the Container App"
  value       = "https://${azurerm_container_app.marketplace_backend.ingress[0].fqdn}"
}

output "container_app_environment_name" {
  description = "Name of the Container App Environment"
  value       = azurerm_container_app_environment.marketplace.name
}

# Static Web App outputs removed - created manually

output "database_host" {
  description = "PostgreSQL server hostname"
  value       = azurerm_postgresql_flexible_server.marketplace.fqdn
}

output "database_name" {
  description = "PostgreSQL database name"
  value       = azurerm_postgresql_flexible_server_database.marketplace.name
}

output "resource_group_name" {
  description = "Name of the resource group"
  value       = azurerm_resource_group.marketplace.name
}

output "storage_account_name" {
  description = "Name of the Azure Storage Account"
  value       = azurerm_storage_account.marketplace_storage.name
}

output "postgresql_server_name" {
  description = "Name of the PostgreSQL server"
  value       = azurerm_postgresql_flexible_server.marketplace.name
}

output "function_app_name" {
  description = "Name of the Azure Function App"
```

```
output "function_app_name" {
  description = "Name of the Azure Function App"
  value       = azurerm_linux_function_app.ai_description.name
}

output "function_app_url" {
  description = "URL of the Azure Function App"
  value       = "https://${azurerm_linux_function_app.ai_description.default_hostname}"
}

output "function_app_hostname" {
  description = "Hostname of the Azure Function App"
  value       = azurerm_linux_function_app.ai_description.default_hostname
}
```

Figure 12. Outputs.tf code

Variables.tf



```
variable "environment" {
  description = "Environment name (dev, staging, prod)"
  type        = string
  default     = "dev"
}

variable "location" {
  description = "Azure region for resources"
  type        = string
  default     = "Southeast Asia"
}

variable "db_admin_username" {
  description = "PostgreSQL administrator username"
  type        = string
  default     = "marketplace_admin"
}

variable "db_admin_password" {
  description = "PostgreSQL administrator password"
  type        = string
  sensitive   = true
}

variable "db_name" {
  description = "PostgreSQL database name"
  type        = string
  default     = "marketplace"
}

variable "storage_account_name" {
  description = "Azure Storage Account name"
  type        = string
}

variable "storage_container_name" {
  description = "Azure Storage Container name for images"
  type        = string
  default     = "images"
}

variable "container_image_tag" {
  description = "Docker image tag for the container"
  type        = string
```

```
variable "container_image_tag" {
  description = "Docker image tag for the container"
  type        = string
  default     = "latest"
}

variable "gemini_api_key" {
  description = "Gemini API key for AI description generation"
  type        = string
  sensitive   = true
}

variable "content_safety_endpoint" {
  description = "Azure Content Safety API endpoint"
  type        = string
  default     = "https://image-content-moderation.services.ai.azure.com"
}

variable "content_safety_key" {
  description = "Azure Content Safety API key"
  type        = string
  sensitive   = true
}
```

Figure 13. variables.tf code

Appendix B – Pipeline YAMLs

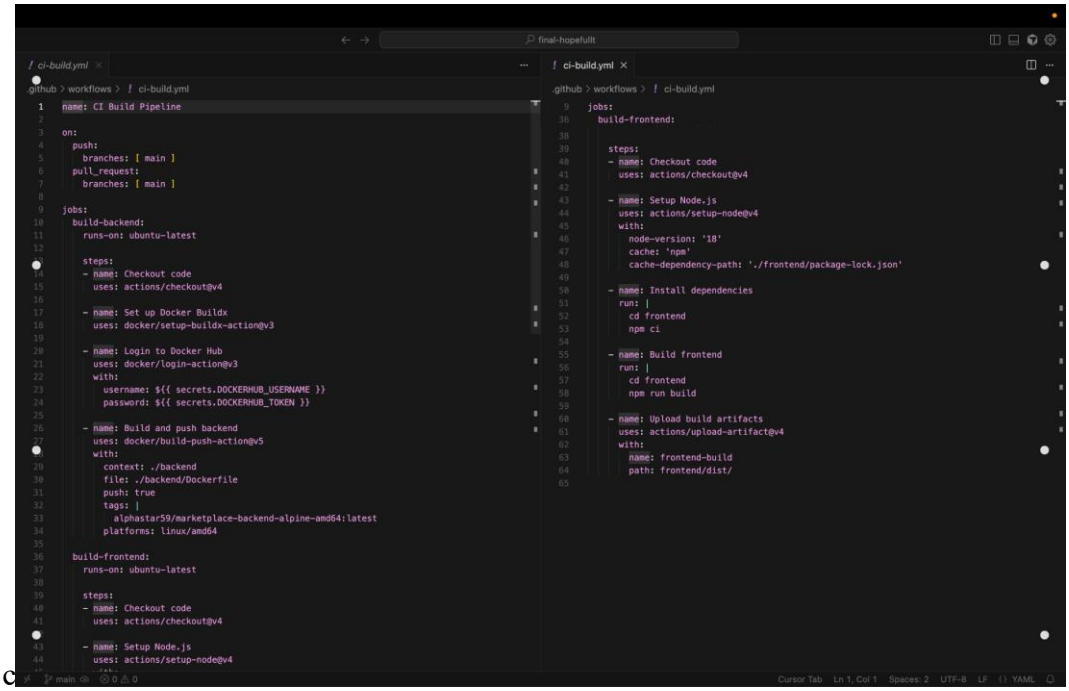


Figure 14. Main Branch Workflow file

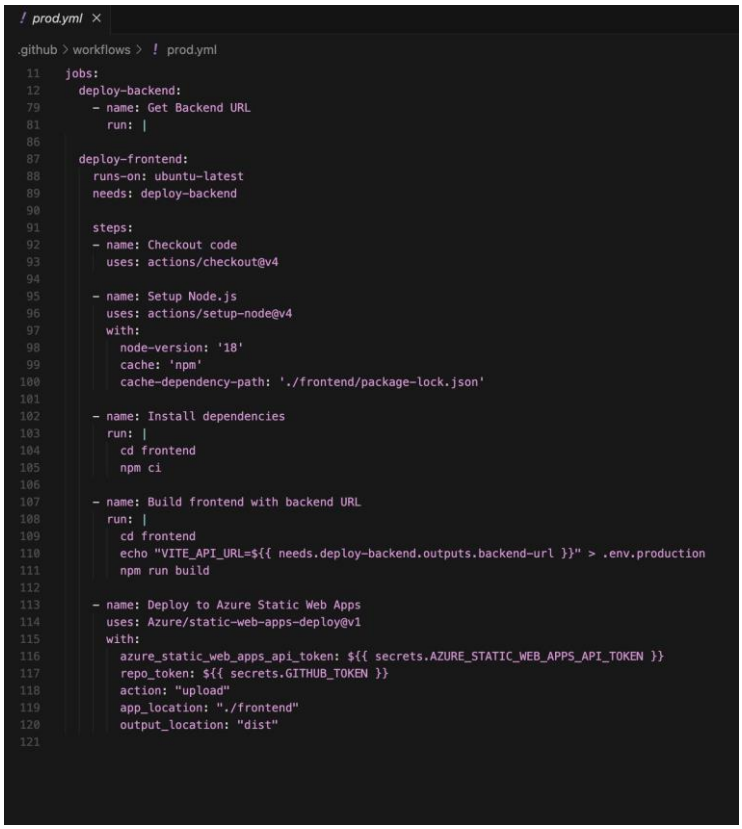


Figure 15. Prod Branch Workflow file

Appendix C – Screenshots (Deployments, AI Integration, Security Scans)

Successful ci/cd run:

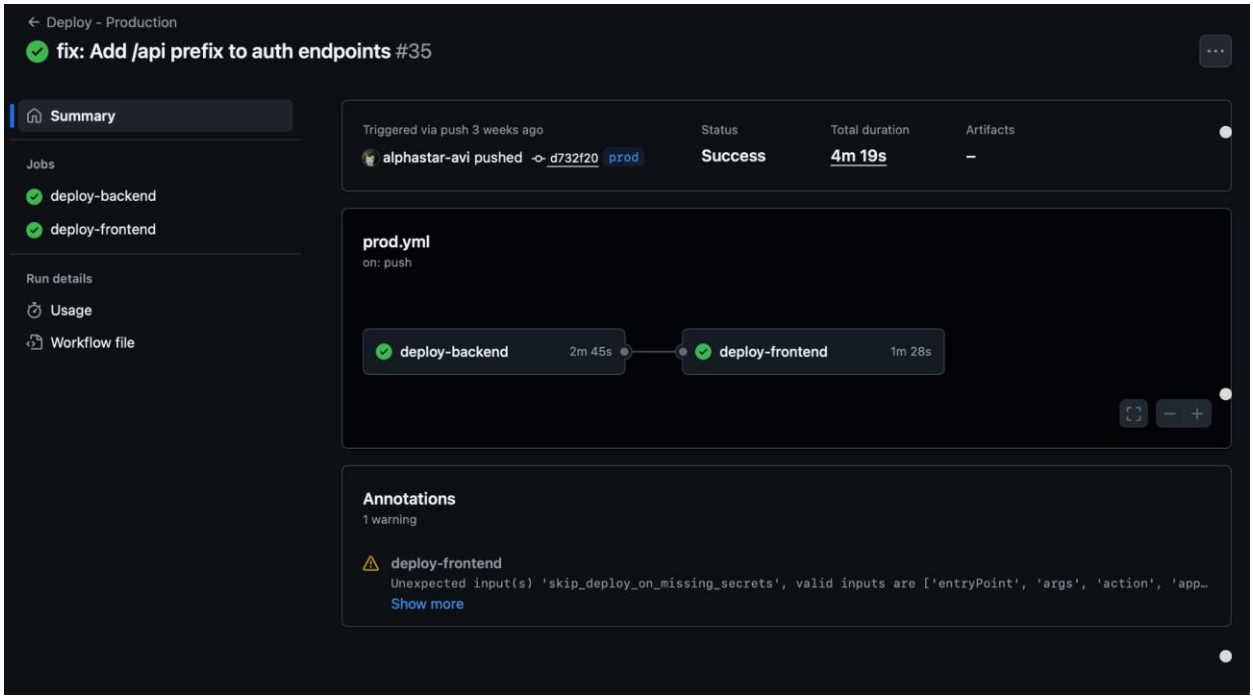


Figure 16. Workflow Showing Frontend and Backend Successfully Deployed

Integrated AI services:

1. **Azure content moderation**

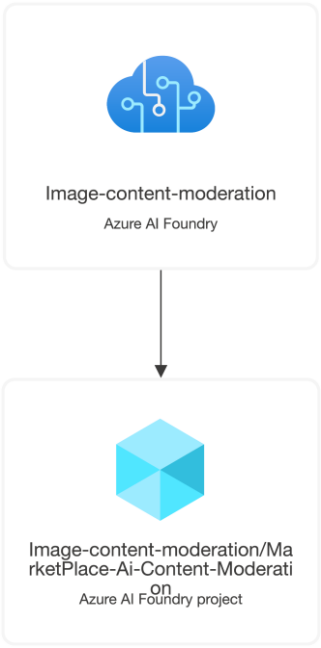


Figure 17. Azure content moderation

2. Azure Container Apps

Figure 18. Description Generation with gemini vision pro

Deployment resource visualizer

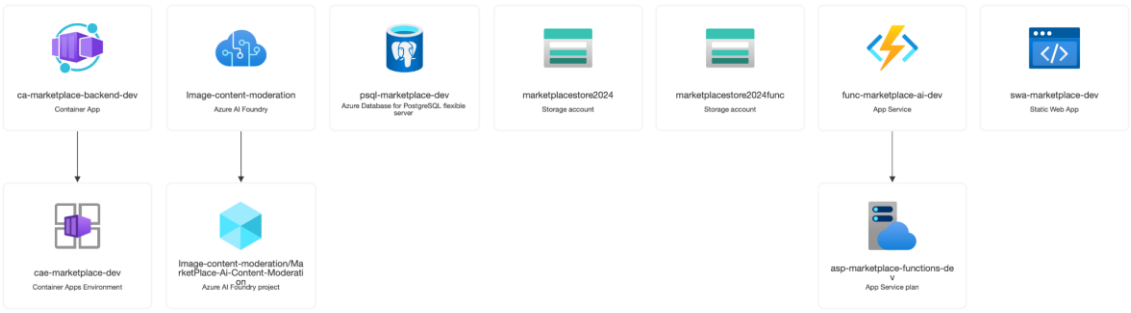


Figure 19. Network Visualizer

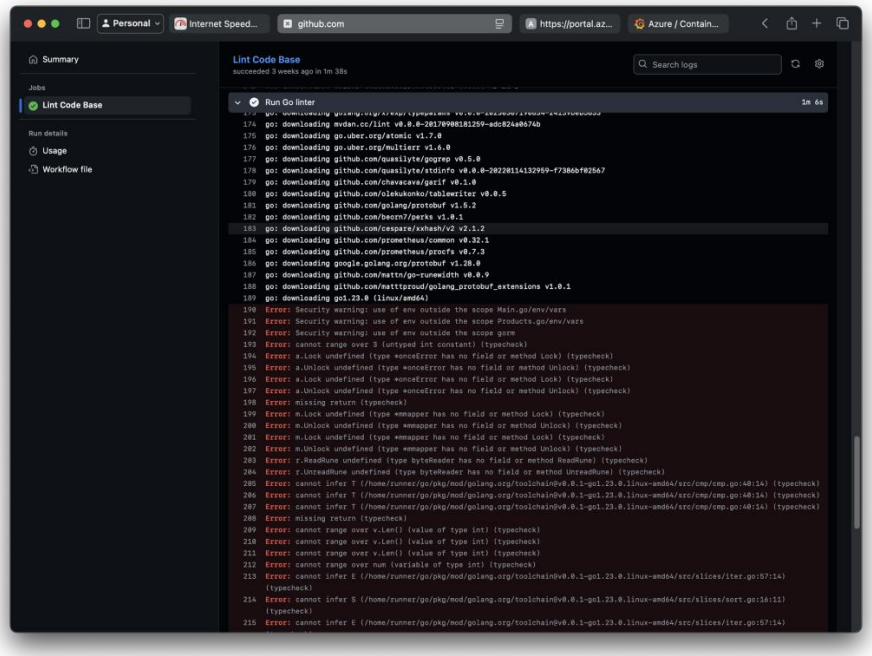
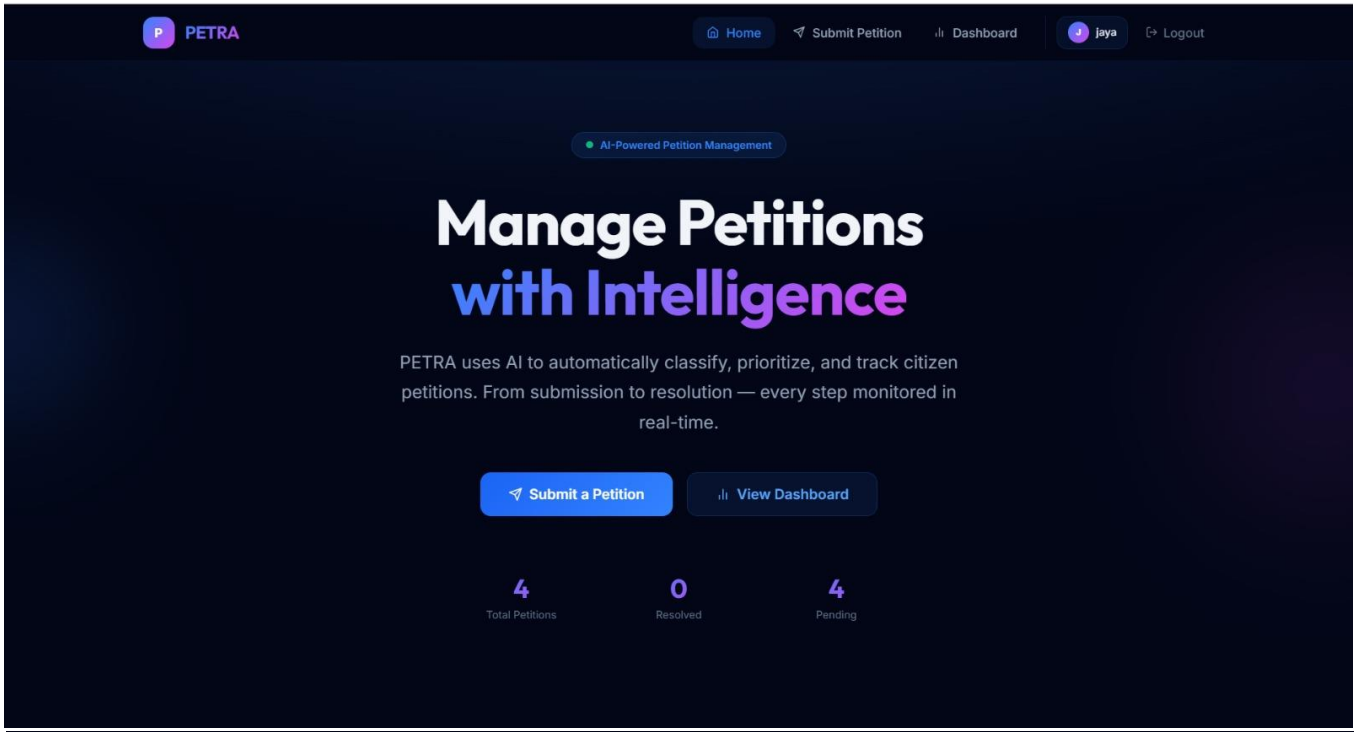
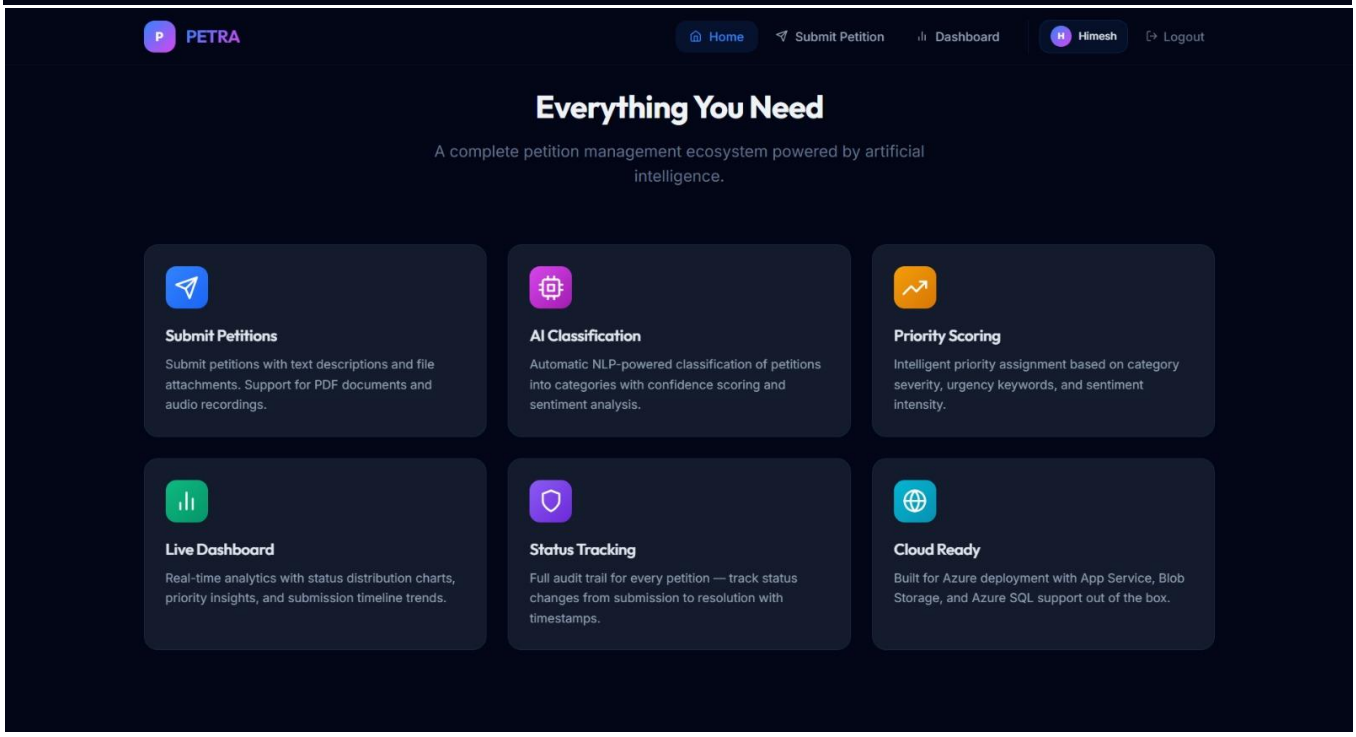
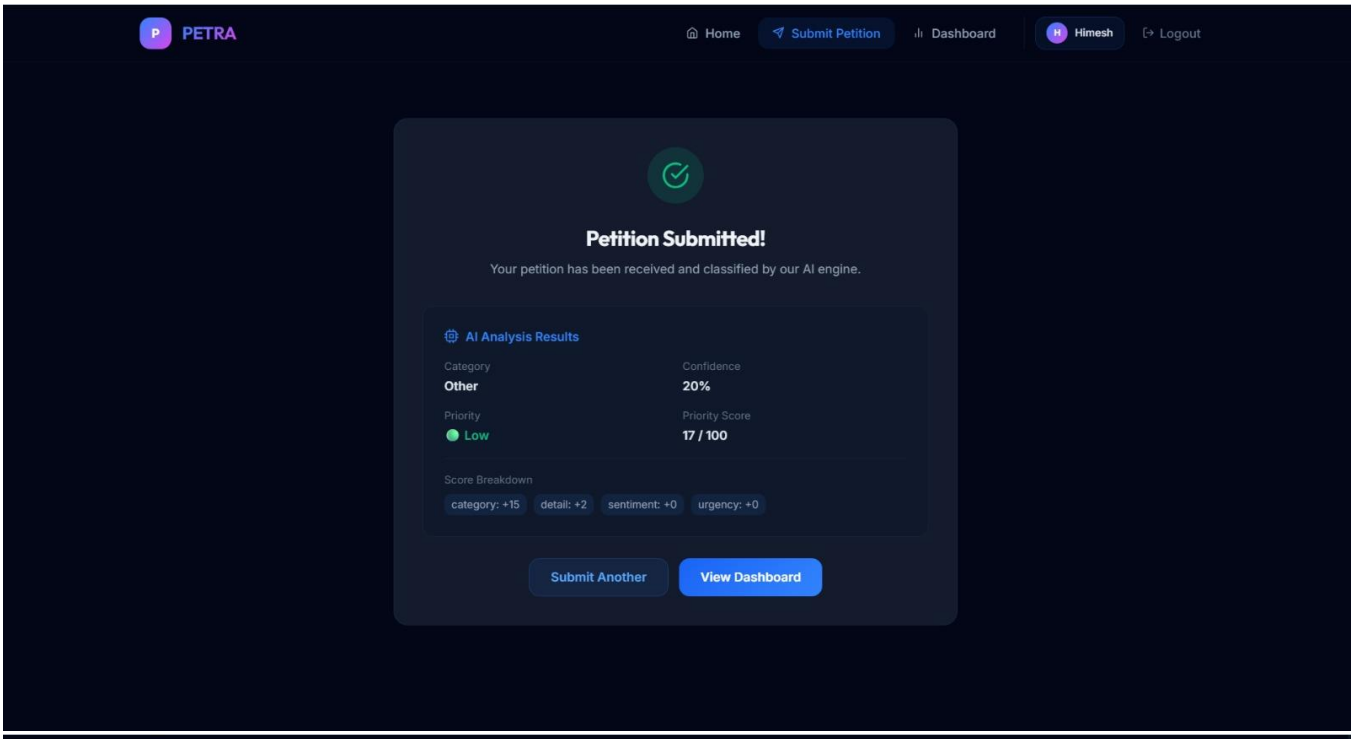


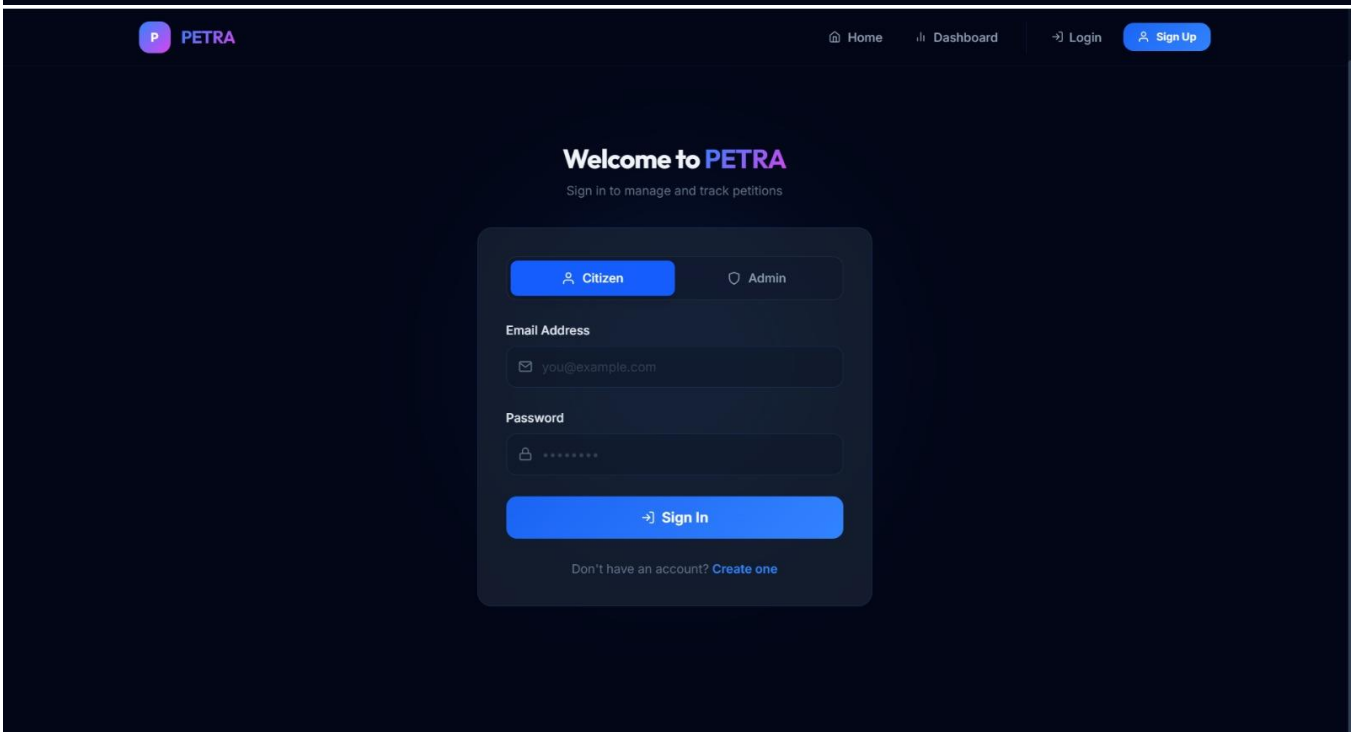
Figure 20. Link Scans

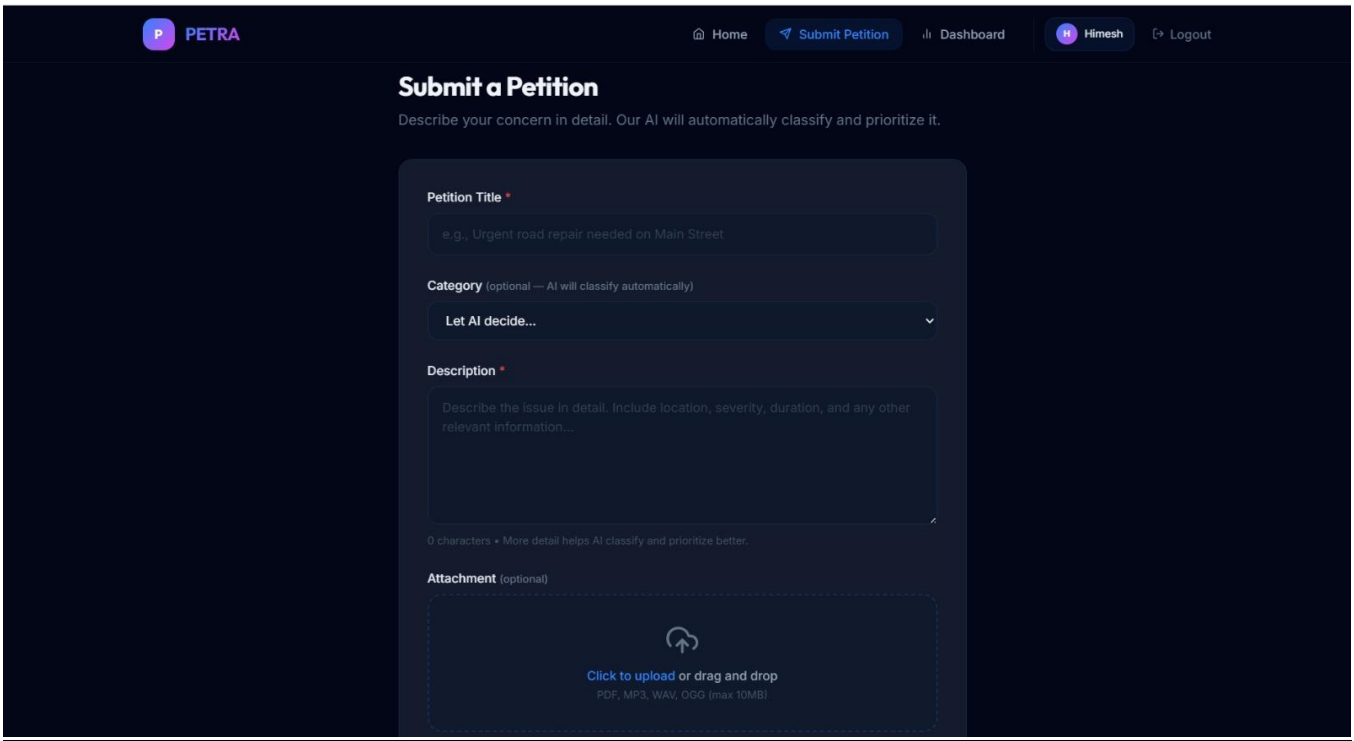
Project Interface

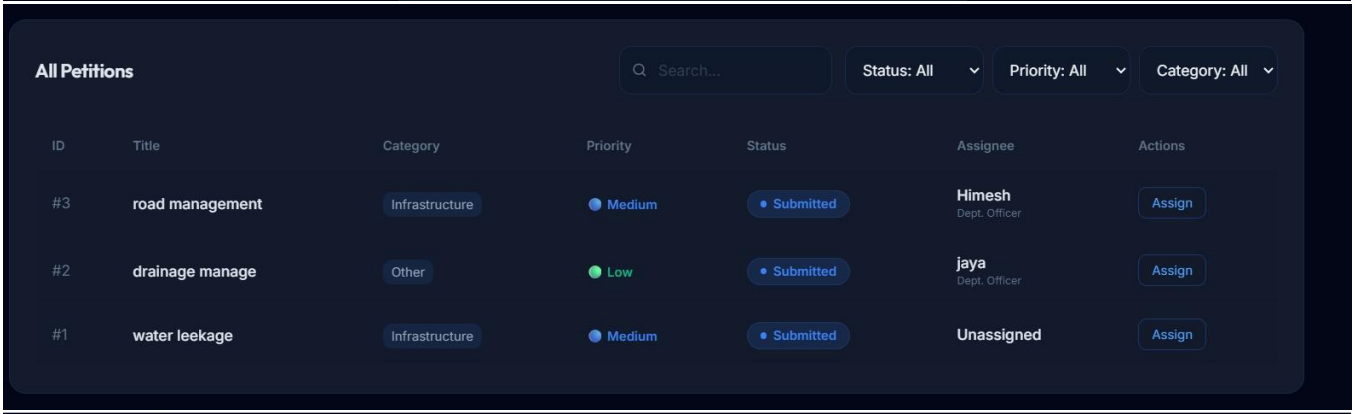


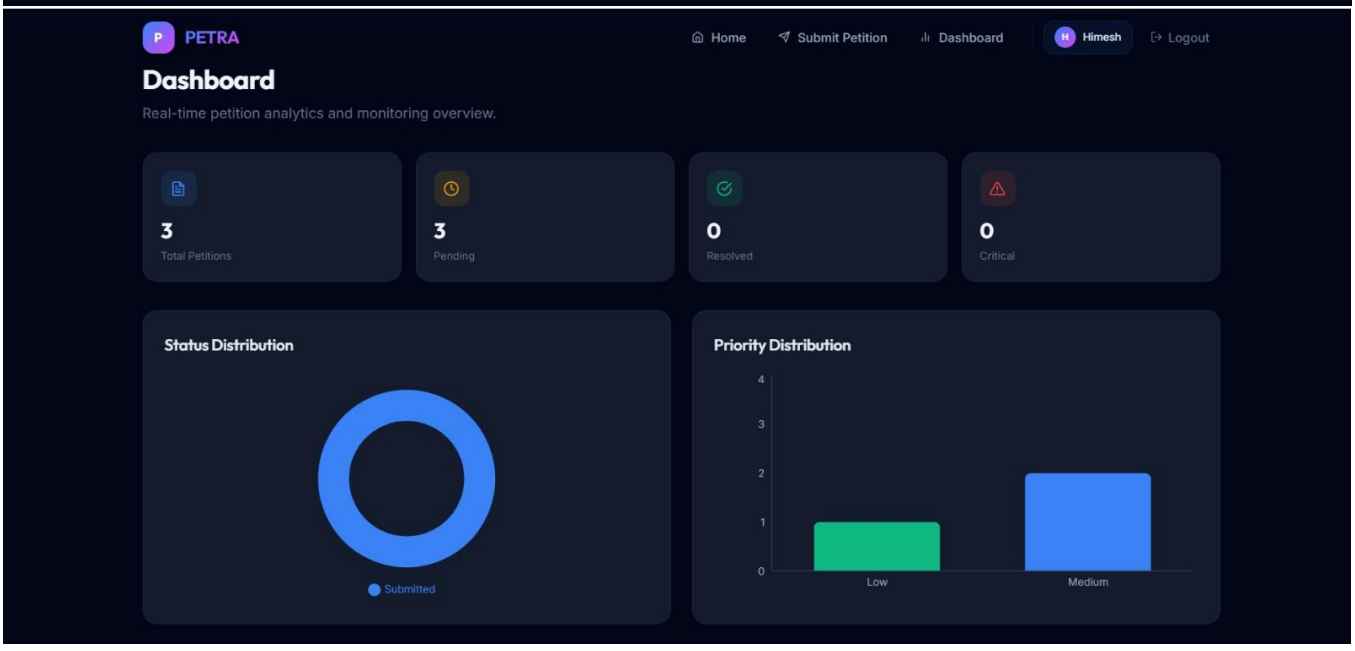



PETRA
Home
Submit Petition
Dashboard
Himesh
Logout
Petition Submitted!
Your petition has been received and classified by our AI engine.
AI Analysis Results
Category
Other
Confidence
20%
Priority
Low
Priority Score
17 / 100
Score Breakdown
category: +15
detail: +2
sentiment: +0
urgency: +0
Submit Another
View Dashboard


PETRA
Home
Dashboard
Login
Sign Up
Welcome to PETRA
Sign in to manage and track petitions
Citizen
Admin
Email Address
you@example.com
Password
Sign In
Don't have an account? Create one


PETRA
Home
Submit Petition
Dashboard
Himesh
Logout
Submit a Petition
Describe your concern in detail. Our AI will automatically classify and prioritize it.
Petition Title *
e.g., Urgent road repair needed on Main Street
Category (optional — AI will classify automatically)
Let AI decide...
Description *
Describe the issue in detail. Include location, severity, duration, and any other relevant information...
0 characters • More detail helps AI classify and prioritize better.
Attachment (optional)
Click to upload or drag and drop
PDF, MP3, WAV, OGG (max 10MB)


All Petitions
Search...
Status: All
Priority: All
Category: All
ID
Title
Category
Priority
Status
Assignee
Actions
#3
road management
Infrastructure
Medium
Submitted
Himesh
Dept. Officer
Assign
#2
drainage manage
Other
Low
Submitted
jaya
Dept. Officer
Assign
#1
water leekage
Infrastructure
Medium
Submitted
Unassigned
Assign


PETRA
Home
Submit Petition
Dashboard
Himesh
Logout
Dashboard
Real-time petition analytics and monitoring overview.
3
Total Petitions
3
Pending
0
Resolved
0
Critical
Status Distribution
Submitted
Priority Distribution
4
3
2
1
0
Low
Medium